

frontmatter

The complete product plan

Amit — this is everything, in the order it makes sense to read it. We start with what people are actually struggling with, then what already exists in the market, then what we have that nobody else does, then what I think we should build and why. The critique is not at the end as a formality; it runs through the whole thing, because a lot of what we believed six weeks ago turned out to be wrong and I would rather you see that than be sold to. Every number here came from a source I opened myself. Where we were wrong, I have said so plainly and shown the correction.

KEY FIGURES



PART I

What is actually broken

1 The problem, in one page

- People now write a lot of their documents and code with an AI. That part works. What does not work is everything around it.
- You cannot tell what the machine wrote.** A week later, looking at a spec or a README, nobody knows which paragraphs were typed by a person and which were generated and never read properly.
 - You cannot hand the work over.** Starting a new session means re-explaining everything. Working with a teammate means they inherit decisions with no record of why.
 - The output quality is dropping in a specific way.** People call it "slop" — technically correct, verbose, generic, and unmaintainable. It passes review because reviewing it properly takes longer than writing it did.

We measured this rather than assumed it. Across the complaint corpora we mined:

EXHIBIT 1
What people complain about, counted

Rank	Complaint	Share	Model or workflow?
1	Sync and losing work across devices	16.5% of loved/hated mentions	WORKFLOW
2	Output quality / slop / unmaintainable	23.7% of AI complaints, +149% in 20 months	WORKFLOW
3	Price	15.7% of weekly posts	—
4	Losing context between sessions	high, but declining	MODEL
5	Hallucination, wrong file edits	declining 19–67% in 20 months	MODEL

NOTE
That last column is the most useful thing in this document. A "model" problem gets fixed by Anthropic or OpenAI within a year or two, for free, and building for it is throwing money away. A "workflow" problem is the mess around the model, and it is ours to solve. Slop is a workflow problem, it is the fastest-growing complaint anywhere in the corpus, and nobody is working on it.

2 What changed, in one table

EXHIBIT 2
Every belief we tested, and what the evidence returned

#	What we believed	Verdict	What the source actually says
1	Byte-exactness makes people switch	REFUTED	4 of 12,556 HN comments · 1 of 3,749 launch comments · once in 1,000 Reddit posts (0.1%) · line endings in 1 of 43,656 GitHub issues · one switch in 11 years
2	Per-hunk review is our demo	REFUTED	Zed docs: "accept or reject each individual change hunk, or the whole set" — I opened the page myself

#	What we believed	Verdict	What the source actually says
3	Handovers and context packs are the wedge	REFUTED	89 launches in 20 months, median 2 points, 88 of 89 never reached 50. Four were called <i>Handoff</i>
4	Decision-flow renders are the obvious first render	REFUTED	563 of 143,283,562 Obsidian downloads. 0.0004%. 3 plugins of 7,079. 0 of 1,600 Reddit posts
5	₹299 is a fair price	REFUTED	Obsidian gives the editor away and charges \$4 for sync alone. We priced a whole editor at \$3.13
6	The engine is the differentiator	CONFIRMED	Cursor staff, three days before I looked: destroyed carriage returns on CRLF is " <i>a known issue we're tracking</i> "
7	The correctness work is real	CONFIRMED	8,513-file pinned corpus, byte-identical verification, 1,575 tests
8	Sync is out of scope	REFUTED	#1 loved feature, #3 hated, #1 switching trigger, 1,251 of 43,656 issues

GOOD

Read row 6 and 7 together. The incumbents have the defect we fixed, they know it, and they have not fixed it. That is a real edge. We were just selling it with the wrong sentence.

PART II

Who this is for, and what the market has

3 What people are actually struggling with

We did not guess at this. We mined Hacker News, Reddit, and GitHub issues and counted, with denominators, so that a number here means something.

EXHIBIT 3

The pain, ranked, with what it is really about

Rank	What they say	How often	Is it the model, or the work around it?
1	"It loses context, I have to re-explain everything"	Highest volume, declining	Model. The labs are fixing it
2	"The output is slop — verbose, generic, unmaintainable"	23.7%, +149% in 20 months	The work around it. Ours
3	"I can't review it fast enough to trust it"	Rising alongside slop	Ours
4	"Sync broke / I lost work"	16.5% of all editor sentiment	Ours
5	"It hallucinated a file that doesn't exist"	Declining 19–67%	Model. Do not build for it
6	"It edited the wrong file"	Declining	Model
7	"It's too expensive"	15.7% of weekly posts in our market	Market condition

The single most useful line in all of our research is that split in the last column. Roughly half of what people complain about is getting better on its own, for free, because Anthropic and OpenAI are spending billions on it. Building a product aimed at hallucination in 2026 is building a product with a two-year shelf life.

The other half is not getting better, because it is not a model problem. It is the mess around the model: what you gave it, what it gave back, who checked, and what happens next week when nobody remembers.

What people do about it today. This matters more than the complaints, because the existing workaround is the real competitor.

- They keep a CLAUDE.md or AGENTS.md file by hand, and it goes stale. We sampled real ones from public repos — most are written once and never updated.
- They paste large chunks of their repo into a chat window and hope.
- They re-read everything the AI wrote, which costs more time than writing it would have.
- Many simply accept the slop, ship it, and deal with it later. This is the most common answer and the most expensive one.

WARN

The uncomfortable one. We looked hard for people complaining that their editor mangled a file. In 12,556 comments about editors, four. In 1,000 Reddit posts, once. Across 43,656 GitHub issues, the phrase "line endings" appears once. Nobody is asking for byte-exactness. We spent months building an argument for a feature the market has never requested.

4 Who has this problem badly enough to pay

Not everyone with the problem will pay for a fix. These are the people who might.

EXHIBIT 4

The candidate buyers, ranked by how much it costs them

Who	What breaks for them	Would they pay?
The solo builder shipping with Claude Code	Ships slop, finds out in production. High volume, low budget	Maybe, and cheaply
A 2–5 person team with no PM	Nobody knows why a decision was made. Onboarding takes weeks	Yes. This is the wedge
An agency handing projects between people and clients	Handover is the product. A bad one costs a relationship	Yes, and more per seat
A platform or DX team maintaining docs	Docs rot; nobody notices until a customer does	Yes, but slow to buy
A regulated or audited team	Must show who wrote what and when	Yes, and most of all — but hardest to reach
An individual note-taker	Nothing breaks. They are happy with Obsidian	No

We have never chosen between the first group and the last two, and that is a real problem. The developer who ships fast wants speed and will not pay much. The agency and the regulated team want *proof* and will pay properly. They need different products. Building for both is how you build for neither.

5 Market segments, and who we actually serve

EXHIBIT 5

The segments, sized and ranked

#	Segment	Size signal	Pain	Will they pay?	Verdict
1	Developers using an AI agent daily	Large and growing fast	Slop, review load	A little, reluctantly	Free tier. Our volume
2	Two-to-five person product teams	Very large	Nobody knows why a decision was made	Yes, per seat	The wedge
3	Agencies and consultancies	Large	Handover <i>is</i> the deliverable	Yes, and more	Second target
4	Docs and DX teams	Medium	Docs rot silently	Yes, slowly	Later, via gates
5	Regulated or audited teams	Smaller	Must evidence who wrote what	Most of all	Hardest to reach. Design for, do not chase
6	Individual note-takers	Huge	None. Obsidian works	No	Not our buyer

The wedge user, specific enough to email ten of this week: a technical founder or lead engineer in a two-to-five person team, shipping with Claude Code or Cursor daily, keeping specs and decisions in a git repo, who has been burned at least once by a document nobody realised was machine-written. They are in *r/ClaudeAI*, *r/ExperiencedDevs* and the Obsidian forum, and they will tell you exactly what they think within an hour of trying something.

6 The two buyers, and why we must choose

This is the deepest problem in our plan and it is not a research gap — it is a decision nobody made.

EXHIBIT 6

The same product, sold to two different people

	The AI-native developer	The person who is liable
Who	Solo builder or small team shipping with Claude Code, Cursor, Codex	Agency, consultancy, regulated team, anyone whose document goes to a client or an auditor
What they feel	"This is slow and I re-read too much"	"If this is wrong, it is my name on it"
What they want	Speed. Fewer keystrokes	Proof. A record of who wrote what
What they pay	Little, reluctantly, and they churn	Properly, and they stay

	The AI-native developer	The person who is liable
How many	Many	Fewer
How to reach them	HN, Reddit, plugin stores, word of mouth	Referral, and almost nothing else
What kills the sale	Any friction at all	No audit trail, no SSO, no entity they recognise

Why we cannot serve both at once. The developer wants provenance to be invisible and fast. The liable buyer wants it to be exportable, signed, and defensible in a dispute. The first is a keyboard shortcut. The second is a compliance feature with a report. Same engine, different products.

My reading, and it is a recommendation not a finding: start with the developer because they are reachable and will tell us quickly whether the idea is any good, and design the provenance record so it *could* become the liable buyer's audit trail later without a rewrite. That is a technical decision we make now — store enough, sign nothing yet.

WARN

If we do not pick, the roadmap will drift toward whoever complains loudest, which is always the developer, who is also the one who will not pay.

7 Positioning, and the objections we will actually hear

The positioning statement. For teams building software with AI, frontmatter is the markdown editor that records which bytes the machine wrote — so you can review what matters instead of re-reading everything. Unlike Cursor or Zed, which rewrite whole files and cannot tell you what they changed, we only ever write an exact byte range.

EXHIBIT 7

Objections, and honest answers

Objection	Our answer	Is the answer good enough?
"Obsidian is free and I like it"	So is ours, and it does something Obsidian does not	Yes
"Cursor already shows me diffs"	A diff is a moment. Provenance is a state that persists until someone deals with it	Yes, but it needs demonstrating, not explaining
"I trust my own review"	Then this changes nothing for you, and you should keep your workflow	Honest, and it loses the sale. Correctly
"Another editor to learn"	It reads your existing folder as-is. No import, no migration, no new format	Yes
"What if you shut down?"	Your files are your files. There is nothing to export because nothing was ever taken	Our strongest answer
"Why should I trust byte-exactness?"	There is a test, it runs on 8,513 real files, and it fails if one byte changes	Yes
"Zed has per-hunk review already"	It does. It also has no idea what a wikilink or a frontmatter key is	Adequate, not crushing
"I don't have this problem"	Then we are not for you today	The right answer, and we should give it

8 What happens if we stop

A shutdown promise is normally marketing. Ours is architectural, which means it holds whether or not we are around to honour it.

EXHIBIT 8

The promises, and whether we could break them

#	Promise	Mechanism	Could we revoke it?
1	Your files stay plain files in folders you chose	The filesystem	No. Not for anything already written
2	No proprietary sidecar carrying meaning you would lose	Delete our files; your documents still work	No
3	Byte-preserving edits	Hash, edit, diff — testable per release	No. Falsifiable on every build
4	Your git history is yours	It is in your repo	No
5	Provenance data is readable without us	Plain text sidecar, documented format	No, provided we document it — and we will

Why this is a commercial argument, not just an ethical one. The single most common objection to a small vendor is "what if you disappear". Our answer is not a promise, it is a property of the design: there is nothing to take back. That is worth more in a B2B conversation than any feature.

9 What the market has today

We tore down every serious tool in two categories — the markdown editors and the AI coding tools — by opening their docs, pricing pages and changelogs.

EXHIBIT 9

The landscape, and where each one stops

Tool	What it is	What it does well	Where it stops
Obsidian	The category leader. Local markdown files	Free, beloved, huge plugin ecosystem	No real AI editing. Sync is a paid add-on and it has duplicated file sections
Notion	The default for teams	Collaboration, databases, polish	Files are not files. Its own export API is named <code>blocksToMarkdownLossy</code>
Cursor	AI-first code editor	Best-in-class agent editing for code	Rewrites whole files. Staff publicly acknowledge destroying carriage returns on CRLF
Zed	Fast editor with an agent panel	Ships per-hunk accept/reject already	Code-first. No markdown vault semantics — no wikilinks, tags, or frontmatter
Claude Code / Codex	Terminal agents	Where the work actually happens now	No document surface at all. You read diffs in a terminal
GitBook / Mintlify	Docs-as-code for teams	Publishing, teams, review	Not an editor you write in. Reliability complaints

Three things this told us that we did not expect.

- **Zed already ships the demo we had planned as our flagship.** Their documentation says: *"You can accept or reject each individual change hunk, or the whole set of changes made by the agent."* I opened the page. That was sixteen of our thirty-eight planned MVP points, building something a funded competitor has already shipped.
- **The incumbents genuinely have the corruption defect we fixed.** Cursor's own staff described doubled blank lines and destroyed carriage returns on CRLF files as *"a known issue we're tracking"* three days before I looked. They know. They have not fixed it.
- **In this category the editor is free.** Obsidian gives the whole editor away and charges \$4/user/month for sync alone. We had planned to charge ₹299 (~\$3.13) for the entire editor — less than one competitor charges for a single add-on.

10 Where each competitor stops, in detail

We opened every one of these — docs, pricing, changelogs — rather than describing them from memory.

EXHIBIT 10

The teardown

Tool	Price today	How it writes files	What it cannot do
Obsidian	Editor free . Sync \$4/user/mo. Publish per site	You write; it does not generate	No AI editing worth the name. Its own sync has duplicated file sections
Notion	~\$10/user/mo	Blocks in a database, exported on request	Files are not files. Its export function is literally named <code>blocksToMarkdownLossy</code>
Cursor	~\$20/user/mo	Whole-file rewrite	Staff acknowledge destroying carriage returns on CRLF. Code-first; markdown is an afterthought
Zed	Free, AI usage billed	Per-hunk accept/reject — already shipped	No vault semantics. No wikilinks, tags or frontmatter awareness
Claude Code	Inside a \$20–200 plan	Diffs in a terminal	No document surface. You are reading patches, not writing
GitHub Copilot	\$10–39/user/mo	Suggestions and PRs	Not a writing surface
Logseq / Reflect / Craft	\$0–15/user/mo	Manual	No AI provenance, no byte guarantees
GitBook / Mintlify	\$6.70–\$300+/mo	Publishing pipeline	Not where you write. Complaints cluster on feature gaps, not reliability

What the price column tells us, and it is not what we assumed.

- The editor itself is worth ₹0 in this category. Obsidian proved it and everyone has priced accordingly.
- What people *do* pay for is **sync** (\$4) and **teams** (\$50/user/year in Obsidian's commercial licence).
- Developers already spend \$20/month on Cursor and \$20–200 on Claude. Our ₹299 (~\$3.13) does not read as good value; it reads as **not serious**.

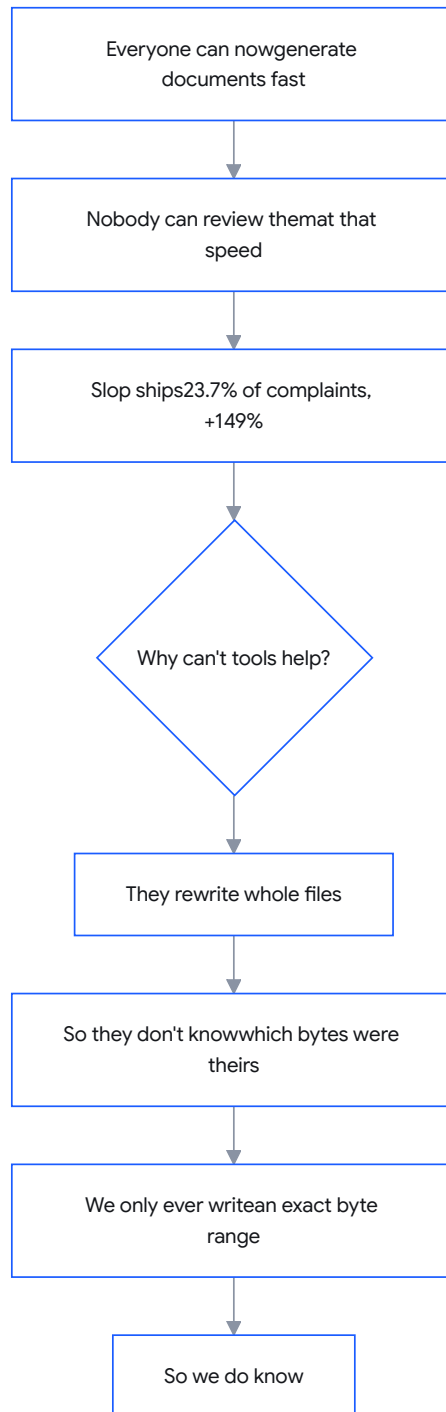
RISK

The finding that changed our plan most. Zed's documentation says: *"You can accept or reject each individual change hunk, or the whole set of changes made by the agent."* That was our flagship demo — sixteen of thirty-eight planned points. It is shipped, documented, and free. We would have found this out at launch.

11 So where is the actual opening

Three facts, laid next to each other:

- Every AI editor **rewrites whole files**, which is why they cannot tell you what they changed and why they corrupt line endings.
- The one that reviews per-hunk is a **code** editor with no idea what a wikilink or a frontmatter key is.
- The fastest-growing complaint in the market — slop, 23.7% and rising — is about **not being able to review output fast enough to trust it**.



GOOD

The opening: be the editor that can tell you which parts a machine wrote. No tool that regenerates files can follow us there, because they genuinely do not have the information.

PART III

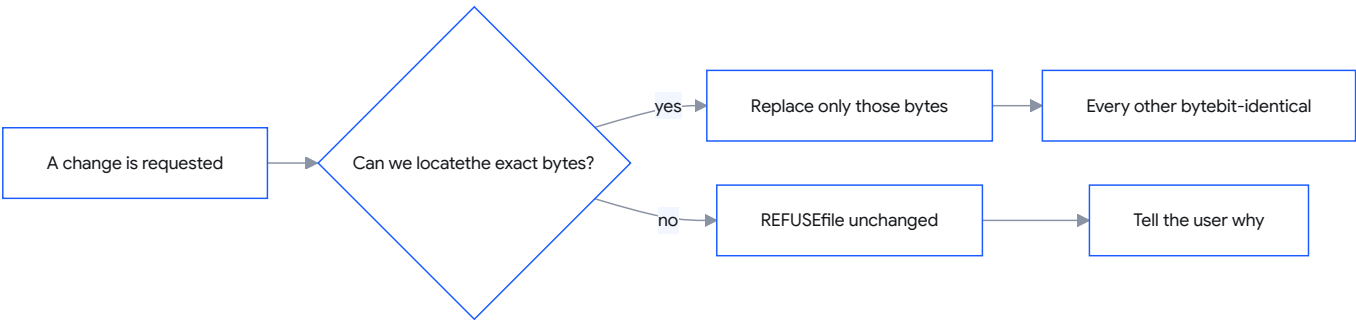
What we already have

12 The engine, in plain terms

The thing we have built and that nobody else has is a way of editing a file that changes only the exact bytes you asked to change, and refuses when it cannot be certain.

How everyone else does it. Parse the file into a tree, modify the tree, write the whole file back out. This is why Notion's export is lossy, why Cursor destroys line endings, and why block editors mangle files. The rewrite touches everything, so any imperfection in the parser becomes a change in your document.

How ours does it. Find the byte range the change applies to. Replace exactly those bytes. Every other byte in the file is bit-identical afterwards. If the range cannot be located unambiguously, refuse and change nothing.



Why this is hard and therefore worth something. Getting it right means handling every way a markdown file can be strange: Windows line endings, byte-order marks, tabs, unusual YAML, non-English text, files written by other tools. We tested it against 8,513 real markdown files pulled from seven strangers' public vaults — pinned by checksum so the test cannot drift — and it does not corrupt them.

EXHIBIT 11

What is already built and proven

Source	25,407 lines of TypeScript across 226 files
Tests	1,575 tests in 98 files
Test corpus	8,513 real markdown files, byte-pinned by checksum
Corpus result	Zero corruption, zero crashes
Shared with sgnk-md	202 of 228 files byte-identical — this is not a new codebase

WARN

What is not built, and we should be honest about it. No CI at all, in a product that wants to sell document checking. Four of our own quality gates have reported "green" while actually being blind. One symbol from one of thirteen engine files reaches the product code — the engine is largely not wired in yet. And 83% of foreign vaults are currently *refused* because of one bug, which means the front door does not open for most people.

13 How the engine actually works

Worth understanding properly, because everything we are selling rests on it.

The contract, in one sentence: locate the exact byte range, replace only those bytes, leave every other byte in the file bit-identical — and if the range cannot be located unambiguously, refuse and change nothing.

What that rules out. We never parse the document into a tree and write the tree back. That single decision is why we do not have the defects the others have, and it is also why our engine is harder to write than theirs.

EXHIBIT 12

What happens to a file, by tool

Operation	A regenerating editor	Ours
Change one frontmatter value	Rewrites the whole block; may reorder keys, requote strings, normalise dates	Replaces the value's bytes. Key order, quoting and comments untouched
Change one table cell	Rewrites every row; may re-pad columns and change line endings	Replaces the cell's bytes
Windows line endings	Frequently normalised to Unix, silently	Preserved exactly
A file with no trailing newline	Usually gains one	Stays as it was
An ambiguous target	Guesses, and usually gets it right	Refuses

The proof it works. We pulled 8,513 real markdown files from seven strangers' public vaults, pinned every one by checksum so the test cannot silently drift, and ran the writer over all of them. Zero corruption, zero crashes. The checksum pinning matters more than it

sounds: it means the test fails if a single byte anywhere in the corpus changes, so we cannot accidentally "fix" a test by changing the data.

Where it is currently broken, and this is the honest part.

Defect	What happens	Effect
NF-1	A list item at column zero in frontmatter is not recognised as belonging to the key above it	83% of real vaults are refused. The most common way of writing a YAML list
NF-2	A closing bracket at column zero, same family	Smaller share of the same problem
NF-3	A frontmatter fence ending in a bare carriage return is not recognised	The writer thinks there is no frontmatter and adds a second block. Structurally destructive
NF-4	Keys with spaces or non-English characters cannot be addressed	942 files have at least one. A fix design exists that recovers 936 of them

WARN
Read NF-1 again. Eighty-three percent of the vaults we tested are refused today, for one bug, with a known fix. That is not a research problem or a design problem. It is four days of work standing between us and a product that opens.

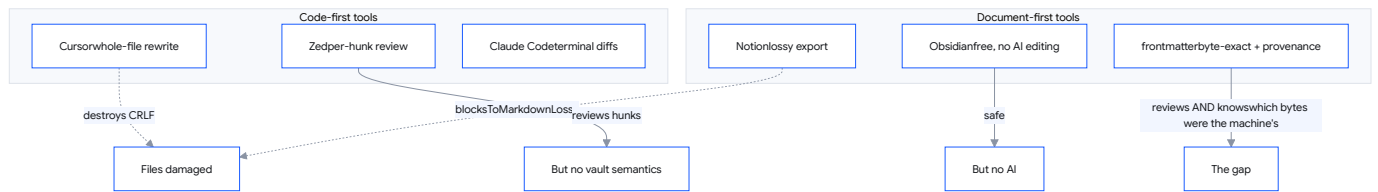
14 Why refusing is the hard part

Every other tool guesses. Guessing looks better in a demo and is worse in practice.

- The industry-standard fuzzy patch library, run here: applied the same patch twice and produced corrupted text — **and reported success both times.** It also applied an edit whose anchoring context no longer existed and reported success. Its own documentation describes this as intended behaviour.
- Git's three-way merge, measured on the same inputs, preserves Windows line endings and a missing final newline, and produces a conflict exactly where a careful writer would want to stop.
- We chose the second behaviour. It is why we can promise what we promise.

The counter-argument, recorded rather than dismissed: conservative merging produces conflicts, and users hate conflicts. That is precisely why competitors ship fuzzy patching. Our mitigation is to make conflict rate a shipped, published metric with a budget — if we are refusing too often, we should know before our users tell us.

15 Where we sit, drawn



Read the diagram as one sentence: the code-first tools review changes but do not understand a markdown vault; the document-first tools understand the vault but either damage files or have no AI at all. Nothing occupies the corner where both are true.

16 What we got wrong

We ran a verification pass over the twenty-one claims the plan leaned on hardest, opening a primary source for each.

EXHIBIT 13

Our own review, against our own case

Verdict	Count	Meaning
CONFIRMED	13	The source says what we said
REVISED	13	Right idea, wrong number or scope
REFUTED	5	False. Deleted from the record
UNVERIFIABLE	3	No openable source exists

Eight of twenty-one survived unchanged. The corrections that matter to you:

#	What we believed	The correction
1	Byte-exactness makes people switch	Four complaints in 12,556. Once in 1,000 Reddit posts. One switch in eleven years

#	What we believed	The correction
2	Per-hunk review is our unique demo	Zed ships it. I opened their docs
3	Generating handovers is the wedge	89 such products launched in 20 months. Median score: 2 points
4	Decision-flow renders are the first thing to build	563 of 143,283,562 Obsidian downloads want it. 0.0004%
5	GST reverse charge starts after ₹20 lakh	No turnover floor. Buying Claude API access starts it at the first rupee
6	Our table editor is fine	It rewrites 3 of 4 lines and destroys 3 of 4 carriage returns. Measured

RISK

I want to be direct about this. **We spent months building an argument for something users do not care about.** The engine is right. The sentence we attached to it was wrong. That is a recoverable mistake and it is much cheaper to find now than after launch — but it is a real mistake and it came from believing our own reasoning instead of checking it.

17 Corrections to the record

EXHIBIT 14

Our own review, against our own case

#	What we believed	The correction	Effect
C1	RCM starts after ₹20 lakh	CGST §24(iii): no turnover floor. Buying Claude API access is importing a service	GST registration starts at the first rupee, not at ₹20 lakh
C2	IT Rules set a 50 lakh threshold	The Rules carry no number — "as notified by the Central Government"	The threshold is not ours to plan against
C3	EU representative under GDPR Art. 27	DSA Art. 13 is a separate mandate with no small-enterprise exemption	Different obligation, different cost
C4	Vanta/Drata charge \$7,000–30,000	Neither publishes a price. drata.com/pricing returns 403	Our B2B ROI headline had no source
C5	GitBook's top complaint is lost work	Of 1,190 discussions, 51.3% are feature requests	Moat #2's only external evidence
C6	frontmatter and sgnk-md are siblings	202 of 228 files byte-identical. 0 files exist only in md	They are one codebase. Park md
C7	The corpus holds 7,959 frontmatter files	7,969. Counted four ways	Not definitional. Just wrong
C8	Our table editor is fine	It rewrites 3 of 4 lines and destroys 3 of 4 carriage returns [measured here]	Dead code. Keep the measurement as a test
C9	Ghost text is refused	It ships, defaulting to off , exactly as specified	No contradiction. An agent got this wrong; I checked

NOTE

62% of our load-bearing claims needed correction when someone opened a primary source: 13 confirmed, 13 revised, 5 refuted, 3 unverifiable. Re-derive before quoting anything in the record.

18 What we take from everything else we have built

We have shipped roughly twenty things. The temptation is to combine them. That temptation is how a focused product becomes a platform nobody buys, so the default answer was **no** and each yes had to be earned by a real mechanism — a shared format, a shared engine, or a shared buyer. "They are both AI" is not a mechanism.

EXHIBIT 15

The ecosystem, item by item

Project	Fuses?	The reasoning
sgnk-md	It is the same product	202 of 228 source files byte-identical. Zero files exist only in md. Frontmatter is a strict superset. One codebase; park md, port its CI first
AIOS — skills, hooks, gates, traces	Internal only	How we work, not what we sell. Its learning loop is not running: 6,884 routing decisions in a week produced one feedback label

Project	Fuses?	The reasoning
The gates from AIOS	Yes, later, as a feature	Cross-reference checking, derived-document building, drift detection. These caught real errors in this document set. That is document CI, which is a product
skills-registry	No	Structurally the same problem — one canonical source projected to many surfaces — but a different product and buyer
HQ	Later	Could be the team admin surface eventually. Not now
AEO / content work	Partly	The published-page surface and its SEO is the one owned distribution channel available to us. Worth keeping alive as a channel, not as a product line
CareerOS · Markex · Advox · INW · Brand OS · Travox · GearUp · stock · trade · pdf	No	They share a founder and nothing else. Combining any of them costs focus and buys nothing

What this means practically. Two things move: `sgnk-md` is parked and its CI comes here. The AIOS gates become a product feature much later, when teams pay us to check their documents. Everything else stays where it is, and **anything not being developed still costs us hosting, domains and attention** — worth an explicit decision about what to shut down.

NOTE

The AIOS honesty note. We have built a genuinely large orchestration substrate — 131 skills, 149 scripts, 69 automated gates, 65 days of measured traces. It is impressive and it is *ours*, not a product. Its own measurements say the learning loop is not closing. We should stop describing it as self-improving until it is, and we should not ship it to users who do not have our problems.

PART IV

What we should build

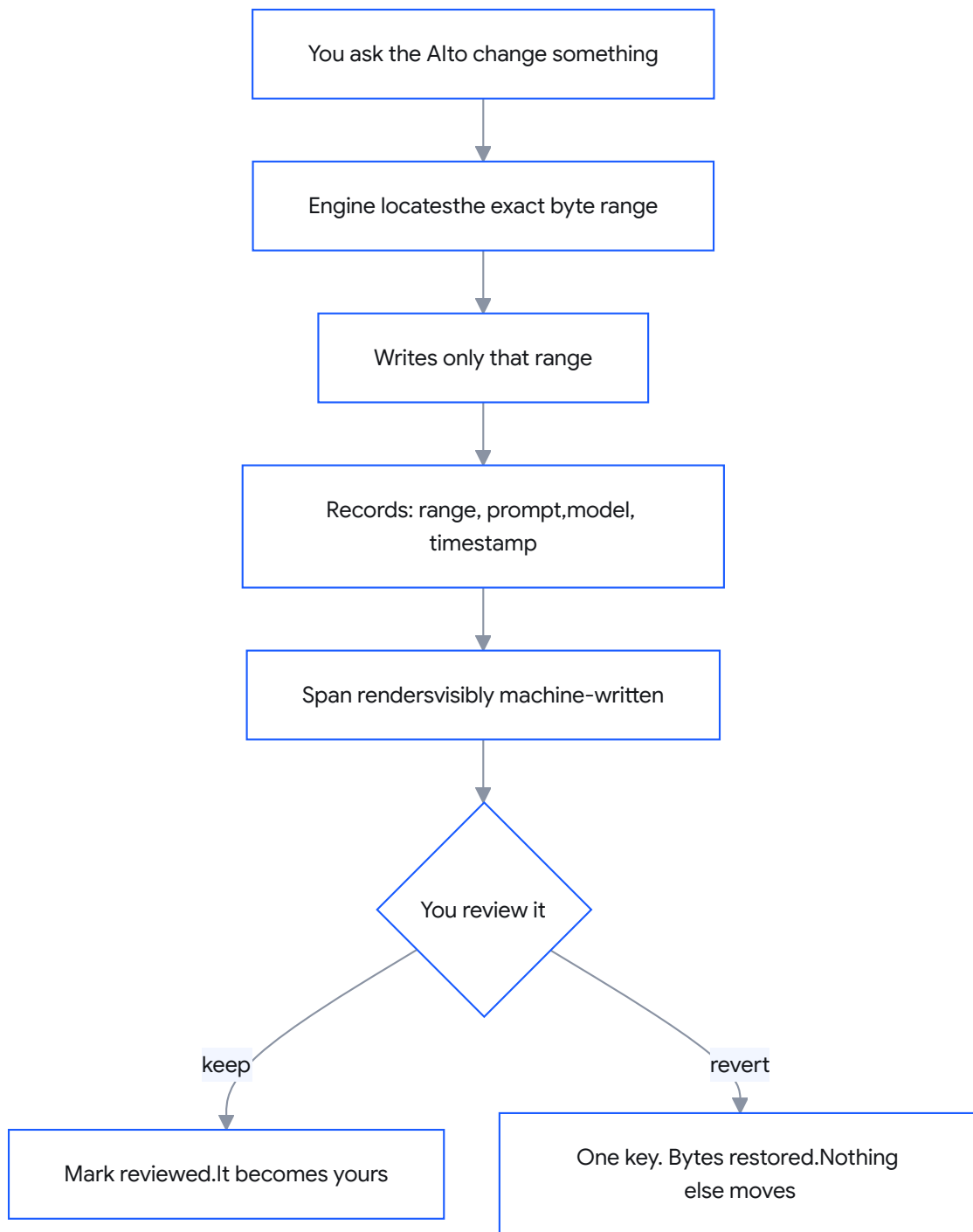
19 The product, in one sentence

GOOD

frontmatter is a markdown editor that knows which bytes a machine wrote. Every AI edit is recorded as a byte range with the prompt, the model and the time. Machine-written spans look different from yours on screen. One keystroke reverts any of them. Nothing else in the file moves.

Why this and not the twenty other things we considered.

- It is the **only** use of our engine that answers a complaint people are actually making, in volume, today.
- **It cannot be copied by anyone who rewrites files.** Cursor and Zed do not know which bytes were theirs, because they regenerate the whole document. We know because we only ever wrote a range.
- It reuses about **90%** of the code that already exists.
- It is a feature of an editor, which is what we have decided to build.



20 The principles, and the things we will never do

Five rules that settle arguments before they start. Each one has a cost, and the cost is stated.

EXHIBIT 16

The principles

#	Principle	What it means in practice	What it costs
1	The file is the only source of truth	Every view is a projection. No view owns state the file does not have	We cannot build features that need hidden state — no comment threads that live only in our database
2	Refuse rather than guess	If we cannot locate a change unambiguously, we change nothing and say why	Some edits fail that a fuzzier tool would complete
3	Never hold the user's documents	Files stay in their repo. We store identity, teams and billing, nothing else	Exit is free, so retention must be earned every month
4	No arbitrary code execution, ever	No plugins, no eval, no user scripts	We refuse the exact moat that made Obsidian unassailable

# Principle	What it means in practice	What it costs
5 Every claim must be falsifiable	If we say "byte-preserving", there is a test that fails when it is not	Slower to ship, and much harder to be wrong in public

The non-goals, stated so nobody proposes them again.

- **Not a project manager.** No kanban, no sprints, no assignees. This is a founder boundary, not a resourcing decision.
- **Not a chat interface.** The user already has an agent. Duplicating it badly helps nobody.
- **Not a plugin platform.** See principle 4. This is the most expensive refusal we make and we make it knowingly.
- **Not a publishing platform,** at least not in v1. It is a permanent, personal, unbounded on-call obligation the day the first stranger publishes.
- **Not a knowledge graph.** Graph views are beautiful and almost nobody uses them twice.

21 How the markdown itself is designed

You asked how the markdown will be designed. This is the most consequential design decision in the product and almost nobody outside engineering thinks about it.

The rule: we add nothing to markdown that breaks it somewhere else. A file we touch must render correctly on GitHub, in Obsidian, in a plain text editor, and in whatever the user opens it with next year.

EXHIBIT 17

How we store things markdown has no syntax for

What we need to store	How we store it	Why not the alternative
Document metadata	YAML frontmatter, the existing convention	Nothing else is universally understood
A prose annotation (a note, a decision card)	A blockquote callout — > [!kind]	It has no closing marker to lose . An unclosed fence swallows the rest of the document; a callout cannot
Opaque data (provenance ranges)	A fenced code block with an info string, written open-and-close in one atomic splice	A fence is unambiguous for data, and writing both ends together means it can never be left open
Where provenance lives	A sidecar file next to the document, in the user's repo	Inline would pollute the prose. A database would mean holding their data
Nothing	Custom syntax, HTML, :: directives	Every one of them degrades on some renderer. We accept :: on input and normalise it away on save

Why the callout decision matters more than it sounds. We tested this across four markdown engines plus GitHub live. An unclosed fence is catastrophic — the CommonMark spec mandates that it swallows everything after it, so one lost closing line destroys the whole document downstream. A callout has no closer to lose. That single measured difference chose our carrier format, and it corrects an earlier decision in our own record.

What a file looks like after we touch it. Identical, except for the bytes you asked to change. Same line endings, same key order, same quoting, same trailing whitespace, same missing final newline if that is how you had it. **If you git diff after an edit, you see only your edit.** That is the whole promise, and it is testable on every release.

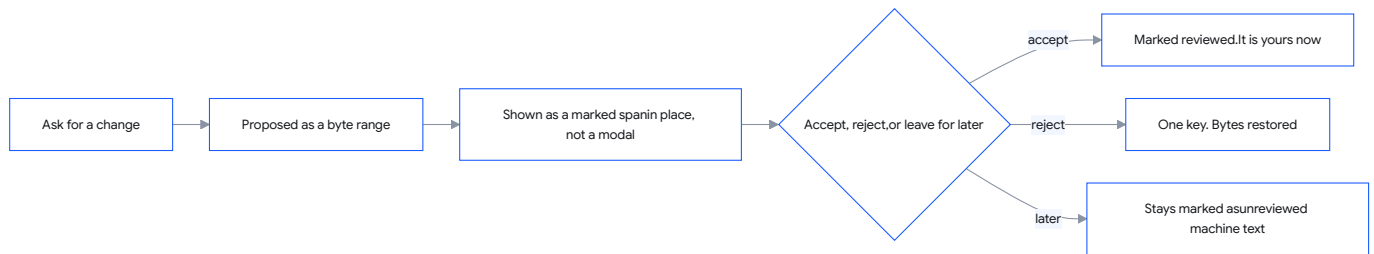
22 How it feels to use

Six flows. Every one names the step most likely to lose the user, because that is where the work actually is.

Flow 1 — first run. Open the app, point it at a folder you already have. It reads it as-is; we impose no structure. You are editing within about thirty seconds.

- *Riskiest step:* the folder read. Today **83% of real vaults get refused** because of one frontmatter bug. That is why NF-1 is the first thing we fix and not a backlog item.
- No sign-up. No account. Nothing to fill in. The single change most likely to hurt adoption is a form on first run.

Flow 2 — an AI edit.



- *Riskiest step:* review fatigue. A forty-hunk change is a chore. Our answer is that you do not have to review it now — unreviewed machine text stays visibly marked until you do. **That is the difference from a diff view: a diff is a moment, provenance is a state.**

Flow 3 — coming back a week later. Open a document. The parts nobody has checked are still marked. You can see at a glance how much of this file has actually been read by a human.

Flow 4 — a vault-wide rename. Rename a tag or a heading. Every file that will change is listed as a reviewable hunk. Anything ambiguous is refused rather than guessed. Accept all, or go one by one.

Flow 5 — two devices. Git merge, with divergence surfaced as a reviewable conflict rather than silently resolved. What we will never do is guess which version you meant.

Flow 6 — a teammate picks it up. They open the repo and can see which parts of the work were machine-written and never reviewed. Today that information does not exist anywhere, in any tool.

23 The onboarding, minute by minute

The first five minutes decide everything, and we have historically under-designed them.

Minute	What happens	What must not happen
0	Open the app. A single button: choose a folder	No sign-up. No account. No email field
0–1	It reads the vault as-is. No import, no migration, no restructure	No "we are indexing your vault" progress bar for minutes
1	You are editing. Everything looks like your files, because they are your files	No onboarding tour
2	You ask the AI for a change. It appears as a marked span	No modal. No diff screen you have to leave the document for
3	You hit one key. It is gone, and <code>git diff</code> proves nothing else moved	Nothing else moved. Ever
5	You realise you can see which parts of this document nobody has read	This is the moment. Everything before it is in service of it

WARN

The single change most likely to hurt us is a form at first run. We collect nothing until there is something to collect it for.

24 The features, and why each one exists

Every feature below names the person who uses it and how often. Anything that could not name one was cut, and the cuts are listed after.

EXHIBIT 18

What ships, and the reason it ships

Feature	Who uses it, how often	Why it exists	Stage
Provenance store and rendering	Anyone reviewing AI output, daily	The product. Answers slop — 23.7% of complaints, nobody else can do it	MVP-0
One-key revert of a machine span	Same person, several times a day	Makes provenance actionable rather than decorative	MVP-0
Engine fixes NF-1/2/3	Everyone, invisibly	83% of vaults are refused today. Five of six user flows start here	MVP-0
Quick-switch, command palette, search	Everyone, constantly	Table stakes. Without them a reviewer stops reviewing	MVP-0
CI, with a deliberate red run first	Us	Four of our gates have reported green while blind	MVP-0
Vault-wide refactor with reviewable diff	Anyone with 100+ notes, monthly	Rename a tag or heading; every affected link shows as a hunk; refuse on ambiguity. 86 likes on this bug	MVP-1

Feature	Who uses it, how often	Why it exists	Stage
Team provenance	Teams reviewing each other's AI work	The first thing worth charging for	MVP-1
Nested-construct live preview	Every Obsidian user, constantly	501 likes — the most-voted bug in the category's history. Our free wedge	MVP-2
Sync, provably safe	Everyone, daily	#1 loved feature, #1 switching trigger, and the only price this category has proven	MVP-2

25 Each feature, specified

Enough detail that you can argue with the design, not just the idea.

F1 · The provenance store

- *What*: every write the engine makes on behalf of an agent is recorded as `{start_byte, end_byte, prompt, model, timestamp, session}`. Stored in a sidecar file inside the user's own repo, not on our servers.
- *Why a sidecar and not a database*: the file is the source of truth. If provenance lived on our servers it would be lost the moment someone cloned the repo elsewhere, and we would be holding user data we promised not to hold.
- *Why it works*: our engine already computes the exact byte range for every write. Nobody else has this information to record.
- *Hard part*: keeping ranges valid as the document is edited around them. This is the anchor problem, and it is the same problem our engine already solves for splices.
- *Who uses it*: invisible. It powers everything else.

F2 · Machine-span rendering

- *What*: text the machine wrote and nobody has reviewed renders with a subtle background. Reviewed text renders normally. Hovering shows the prompt, the model and when.
- *Why subtle and not loud*: a document where half the text is highlighted in yellow is unreadable. This has to be information you can ignore until you want it.
- *Who uses it*: everyone, constantly, without thinking about it.

F3 · One-key revert

- *What*: cursor inside a machine span, one keystroke, the original bytes return. Nothing else in the file moves.
- *Why this and not undo*: undo is chronological and breaks the moment you have typed anything since. This is spatial — revert *that thing*, whenever it was written.
- *Who uses it*: anyone reviewing, several times a session.

F4 · The review state

- *What*: a document knows what fraction of it has been read by a human. A file that is 80% unreviewed machine text looks different in the file tree.
- *Why it matters*: this is the difference between a diff and provenance. A diff is a moment you either handle or lose. **Provenance is a state that persists until someone deals with it.**
- *Who uses it*: the person picking up work a week later, and the teammate inheriting it.

F5 · Vault-wide refactor

- *What*: rename a tag, a heading or a property key. Every file that will change appears as a reviewable hunk. Ambiguous targets are refused, not guessed.
- *Why it belongs to us*: this is a byte-exact operation across hundreds of files. Anyone who regenerates would rewrite every file it touches. The most-liked complaint in this space — 86 likes — is links breaking on rename.
- *Who uses it*: anyone with more than a hundred notes, monthly. Painful when it happens.

F6 · Nested-construct live preview

- *What*: code blocks, quotes and callouts inside list items render correctly while you type.
- *Why it is here at all*: it is the most-voted bug in the category's history — 501 likes, with siblings at 96, 82, 50, 36 and 33. It is not our differentiator. It is the cheapest way to be *noticed* by people who already have this pain.
- *Who uses it*: every Obsidian user, constantly. Which is exactly the point.

F7 · The gates, later

- *What*: automated checks over a repo of markdown — do all cross-references resolve, is anything stale, which sections are unreviewed machine text.
- *Why later*: it is a team product with a team sale, and we should not be selling to teams before we have individuals.

- *Evidence it works:* we run four of these on ourselves and they caught real errors in this document.

EXHIBIT 19

The build order, and the reasoning

Feature	Weeks	Why it is in this stage
MVP-0 Engine fixes + F1 + F2 + F3	10	Without the fixes, 83% of people cannot open the app. Without F1–F3 there is no product
MVP-0 Table stakes: quick-switch, palette, search (inside)		A reviewer who cannot navigate stops reviewing
MVP-1 F4 + F5 + billing + teams	8	The first things worth money, and the first things that need money to exist
MVP-2 F6 free + sync	12	Distribution and the only proven price in the category

26 Why provenance and not the other twenty ideas

We scored nine serious options against evidence of demand, time to revenue, defensibility, fit with our size, and how much of the existing code they reuse.

EXHIBIT 20

The options we considered, scored

Option	Demand	Time to revenue	Reuses engine	Verdict
Provenance in the editor	9/10	~5 months	~90%	Build this
Vault-wide refactor	5/10	~5 months	~90%	Ships with it, MVP-1
Sync, provably safe	9/10	~12 months	~60%	The only proven price. Later
Nested-construct live preview	8/10	Free	~15%	The free wedge. Ship week 2
Document CI for teams	4/10	~6 months	~70%	Later, and it may be the B2B line
Engine as a library / MCP server	4/10	~9 months	100%	Feeds competitors. Not yet
One-time \$49 licence	3/10	~4 months	~90%	A pricing option, not a product
Services, productised	9/10	This month	~10%	Funds everything. Not an asset
Redeploy to another product	6/10	~12 months	~30%	The honest fallback if tests fail

Why provenance wins on the axis that matters. It is the only option where the thing that makes it hard to build — knowing exactly which bytes changed — is a thing we already solved and nobody else has. Every other option on that list could be built by a competitor in a quarter.

27 What we deliberately will not build

A refusal with no cost is not a real refusal, so each one names what we give up.

EXHIBIT 21

The no list

Not building	Why	What we lose
Decision-flow renders	0.0004% of the market wants them	An idea we liked. Nothing else
Degradation certificate as a user feature	Technically lovely, commercially irrelevant	A talking point
Byte-level time travel, two-way renders	Nobody asked	Engineering fun
Project management, kanban, calendars	We are not Notion and cannot win that fight	Some team buyers
Plugins and arbitrary code execution	Ends the corruption guarantee; turns every prompt injection into remote code execution	The ecosystem moat Obsidian has. This is the most expensive refusal on the list
A chat sidebar	Duplicates the agent the user already runs	Nothing. It is table stakes done badly
Mobile, at first	Cost and focus	Real users. Roughly 15% of complaints are mobile

WARN

The plugin refusal deserves a proper argument. Obsidian's moat *is* its plugin ecosystem. Refusing plugins means refusing the thing that made the category leader unassailable. We refuse it because arbitrary third-party code in the editor makes "we never corrupt your file" unprovable, and that guarantee is the entire product. But we should be clear-eyed: this closes the most proven growth path in the category, and we need the free live-preview plugin *in their store* partly to compensate.

28 Simplicity, as something we enforce rather than intend

Every product intends to stay simple. This is how we make it structural.

- **The surface budget.** A new user may meet no more than **seven** top-level concepts in their first session: the folder, the document, the machine span, revert, quick-switch, search, settings. Anything that would be the eighth has to displace one of them.
- **The admission test.** A proposed feature must name the person who uses it and how often. If it cannot, it is cut, and the cut is recorded so nobody re-proposes it in three months.
- **The settings rule.** Every toggle is a decision we failed to make. The settings page is one screen and stays one screen.
- **The projection law does the heavy lifting.** Because every view must be a deterministic projection of the file owning no state, anything requiring its own hidden state is *already* forbidden by the architecture. Most feature bloat is state bloat, and we made that structurally impossible.
- **What this costs us:** we will say no to things customers ask for. Some will leave. That is the trade, and it is the reason the product can stay comprehensible.

29 How the product should be designed

The design job here is unusual: the most important thing on screen is information *about* the text, shown without making the text harder to read.

The governing rule. A document with provenance on must be as readable as one with it off. If a user turns provenance off to read comfortably, we have failed.

EXHIBIT 22

The visual language

Element	Decision	Why
Machine-written, unreviewed	A very light tinted background, no border, no icon	It has to be ignorable. A border or icon fragments the line and destroys reading rhythm
Machine-written, reviewed	Nothing. It renders as normal text	Once you have read it, it is yours. Permanent marking would be noise
Hover	A small panel: prompt, model, when, and a revert control	On demand only. Nothing hovers into view by itself
Document-level state	A thin bar in the file tree showing unreviewed share	Glanceable. Never a number you have to interpret
Refusal	An inline note where the change would have gone, in plain words	Never a modal. A modal for a refusal makes a normal outcome feel like an error
Conflict	Two versions side by side, with what differs marked	The one place we may interrupt, because the cost of guessing is losing work

The typography and the surface. Google Sans for interface, Google Sans Code for the editor, on white. One accent colour. Square corners. Hairline borders. One elevation rule. This is our existing design system and it is already written down — the important thing is that we actually apply it, since the shipped `globals.css` is currently a different system inherited from a sibling project and self-describes as “*Linear-style modern SaaS*”. That is a real inconsistency and it is on the fix list.

Icons: inline SVG only, from one set. Never emoji, never a web font. An icon font that fails to load renders the literal ligature text or a blank box, and it will fail to load on a slow connection or a strict content policy. Inline SVG carries the vector in the markup and renders every time.

What the interface must not become.

- No sidebar of panels. The document is the interface.
- No settings page that grows. Every toggle we add is a decision we failed to make.
- No dashboard. Nobody opens an editor to look at a dashboard.
- No chat window. The user already has an agent; duplicating it badly helps nobody.

30 Every screen, and what it is for

The product is deliberately small. Eleven screens, and four of them are dialogs.

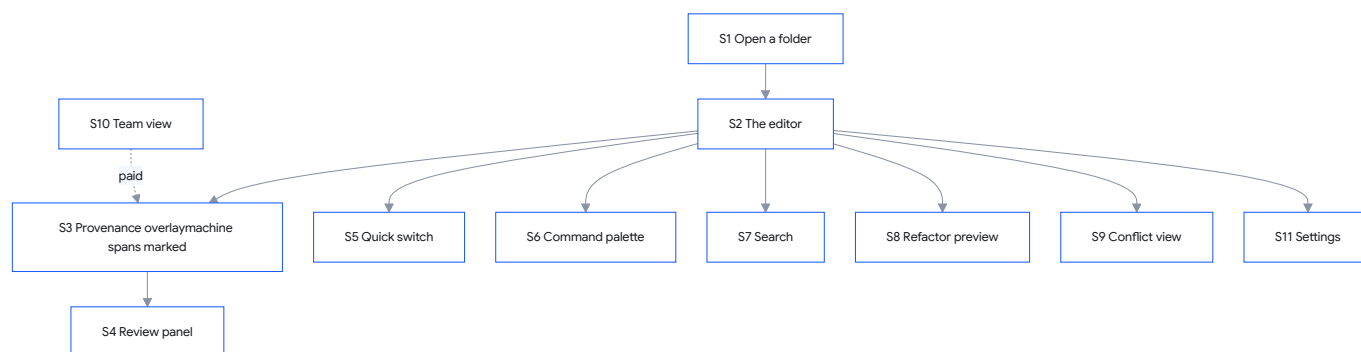
EXHIBIT 23

The screen inventory

#	Screen	What it does	Why it exists	Stage
S1	Open a folder	One button. Pick a directory or connect a repo	The entire first run. No account, no form	MVP-0
S2	The editor	The document, full width. File tree collapsible on the left	This is 95% of the product. Everything else serves it	MVP-0
S3	The provenance layer	Not a screen — an overlay on S2. Machine spans tinted, hover for detail	The differentiator. It must live <i>inside</i> the document, not beside it	MVP-0
S4	Review panel	A list of unreviewed machine spans in this document, keyboard-navigable	For someone catching up on a document they did not watch being written	MVP-0
S5	Quick switcher	Fuzzy file search, keyboard-first	Table stakes. Its absence ends a review before it starts	MVP-0
S6	Command palette	Every action, searchable	Table stakes, and it is how power users learn a product	MVP-0
S7	Search	Across the vault, with results in context	Table stakes	MVP-0
S8	Refactor preview	Rename something; every affected file listed as an accept/reject hunk	The 86-like problem. Our engine's most visible capability	MVP-1
S9	Conflict view	Two versions side by side, differences marked, you choose	The only place we interrupt, because guessing here loses work	MVP-1
S10	Team view	Who wrote what across a shared repo; unreviewed share per document	The paid surface. Nothing else in the product is per-seat	MVP-1
S11	Settings	One short page. Theme, keymap, AI key, provenance on/off	Deliberately small. Every toggle is a decision we failed to make	MVP-0

Four dialogs, and no more: connect a repo · enter an AI key · a refusal explanation · the update prompt.

What we are not building, and it is a long list on purpose: no dashboard, no analytics screen, no template gallery, no plugin browser, no onboarding tour, no chat sidebar, no kanban, no calendar, no graph view, no publish console.



31 What is in each screen

Screen by screen, the actual elements, so this can be designed from.

EXHIBIT 24

S2 — the editor, the 95% screen

Region	What is in it	Behaviour
Left rail, collapsible	File tree. Each file shows a thin bar for unreviewed machine share	Collapses to nothing. Keyboard-toggleable. Remembers state
Centre, always	The document. Full measure, generous line height	This is the product. Nothing overlays it uninvited
Provenance layer	Tinted spans for unreviewed machine text	Toggleable. No borders, no icons, no gutter marks

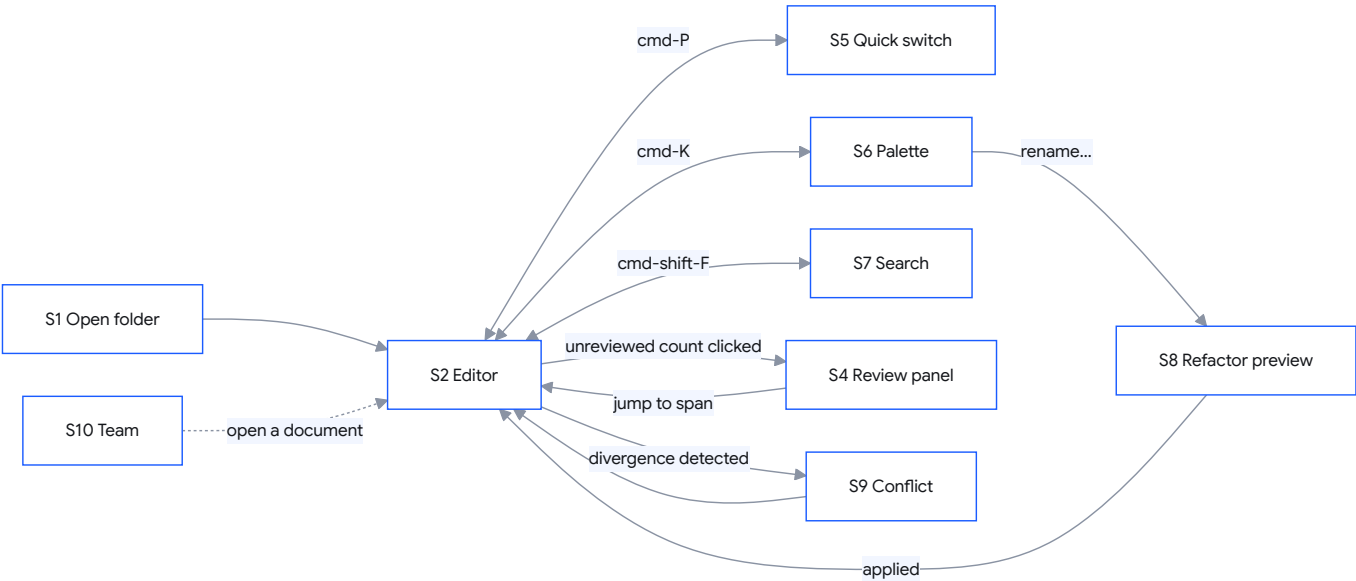
Region	What is in it	Behaviour
Top, minimal	Breadcrumb path, sync state, one overflow menu	No toolbar. Formatting is markdown, typed
Bottom, thin	Word count, cursor position, unreviewed count for this file	Ambient. Never demands attention
On hover over a span	Small panel: prompt, model, timestamp, revert	On demand only, after a delay, dismissible with Escape

EXHIBIT 25

The other screens, and their elements

Screen	Elements
S1 Open a folder	One button. A recent-folders list if any. A line explaining nothing leaves your machine. No account, no email
S4 Review panel	Ordered list of unreviewed spans · each with its first line, model, age · accept / revert / skip · keyboard j-k-a-r · a counter that goes to zero
S5 Quick switcher	Input, fuzzy-ranked results, path shown, recent-first when empty. Enter opens, Escape closes
S6 Command palette	Input, grouped commands, keyboard hints beside each. Shows what a command does before you run it
S7 Search	Query, results grouped by file with two lines of context, count. Replace is a separate deliberate mode
S8 Refactor preview	What is changing, stated in one line · the affected-file list with per-file hunks · accept all / per file / cancel · a refusal list with the reason for each
S9 Conflict view	Two panes, differences marked, a third pane for the result · take-mine / take-theirs / edit · never auto-resolves
S10 Team view	Repo picker · per-document unreviewed share · per-person contribution · a filter for "machine-written, nobody reviewed"
S11 Settings	One page, six groups, no tabs: appearance · keymap · AI provider and key · provenance defaults · git identity · about

32 How the screens talk to each other



Three rules the navigation obeys.

- Everything returns to S2. No screen is a destination; each is a detour that hands you back to the document.
- Escape always goes back one step, and never loses work.
- No modal blocks the document except the conflict view, which blocks because proceeding without a decision would lose data.

33 The components, and who owns each

EXHIBIT 26

The system, by component

Component	What it is	Where it runs	Risk
Splice engine	Locate byte range, replace, or refuse	Client and server, pure functions over bytes	The offset boundary. Bytes vs UTF-16

Component	What it is	Where it runs	Risk
Provenance store	Byte ranges + prompt + model + time, in a sidecar in the user's repo	Client, written to their disk	Keeping ranges valid as text moves around them
Offset map	Translates byte positions to editor positions	Client	Highest-risk seam in the product
Construct detectors	Recognises the 19 markdown constructs we handle	Client and server	Six known defects, all scoped
Editor shell	CodeMirror 6, four modes	Client	Well understood
Repo connector	GitHub App; read, propose, commit	Server	Token scope and consent-screen friction
Control plane	Identity, teams, entitlements, billing. Zero document bytes	Postgres	Nothing sensitive lives here by design
AI gateway	Routes to the user's provider or ours	Server	Cost control lives here
Certificate	Cross-engine render comparison	Server, batch	Internal only for now
Gates	Reference checks, drift detection over a repo	CI	Becomes the team product later

34 How we play with the tools people already use

We are joining a workflow, not replacing one. That means interoperating with things that already have adoption rather than inventing a format and hoping.

EXHIBIT 27

What we interoperate with, and how

Thing	Status	Our position
Plain markdown / CommonMark	The substrate	Everything we write must render correctly everywhere. Non-negotiable
YAML frontmatter	Universal convention	We read and write it byte-exactly, preserving key order and quoting
AGENTS.md / CLAUDE.md	Real, adopted conventions	Read them, respect them, never compete with them. Inventing a rival file would be a mistake
.cursorrules and similar	Tool-specific	Read where useful. Do not write
MCP (Model Context Protocol)	Growing standard	Strong candidate: be an MCP server the user's existing agent connects to. Feeds the agent they already run rather than duplicating it
Git	The transport and the history	We are a git client. We never invent our own versioning
GitHub App	Repo access	The connector, never the identity
Obsidian vault conventions	Wikilinks, tags, properties	Read and preserve. Their users are our audience
Mermaid	Diagrams in markdown	Render. Do not extend
Our own provenance sidecar	New	Documented and plain text. If it is not readable without us, we have broken our own promise

The strategic question inside this table. Do we compete with the terminal agent the user already runs, or feed it? Feeding it is cheaper, and it may be the whole product: frontmatter as the place where the agent's work becomes reviewable, rather than as another place to talk to an agent. **That is the MCP answer and I think it is right**, but it is a decision, not a conclusion, and it belongs in the session.

35 What documents we understand, and where those definitions come from

We do not invent document formats. We render the ones that already exist, and we researched which of those are real standards and which are folklore — because promising to "support ADRs" means nothing if the format is undefined.

EXHIBIT 28

The document canon, with its actual provenance

Document	Is there a real standard?	What we do
ADR	Yes — MADR template and the Nygard structure	Render both. They are the most-used decision formats
RFC (IETF)	Yes — RFC 7322, 23 elements, 10 required	Render. Precise and stable
RFC (Rust)	Yes — 9 sections, a real template	Render
Changelog	Yes — Keep a Changelog 1.1.0	Render. Widely used and unambiguous
OpenAPI	Yes — versioned spec	Recognise, do not render
Gherkin	Yes — Cucumber keywords	Recognise
Postmortem	Partly — Google SRE, but their two books disagree with each other	Render loosely, prescribe nothing
Runbook	No standard body at all	Offer a structure, do not claim a standard
PRD / FRD / TRD	None. Vendor blog templates only	Do not pretend otherwise. Every "PRD standard" claim is unsourced
SRS	Partly — ISO 29148 exists but is paywalled and returned 403	Do not claim conformance we cannot verify

NOTE

Why this table is in a product document. It is the difference between "we support decision records" and "we render MADR and Nygard, and we will not pretend PRD has a standard because it does not". The second is credible to the exact buyer we want. The first is marketing.

36 Accessibility, and text that is not English

Both were absent from our plan and both are cheap now and expensive later.

Accessibility.

- Target WCAG 2.2 AA. It is all-or-nothing — a single failing contrast pair fails the whole conformance claim, so this cannot be retrofitted selectively.
- Our own shipped design tokens currently include a body-faint colour that **fails AA** at a measured 1.984:1 against white. Fixing it is a one-line change and it is on the MVP-0 list.
- Provenance marking must never be *only* a colour. A colour-blind or low-vision user needs the same information from the hover panel and from keyboard navigation.
- Every provenance action must be reachable from the keyboard, because the people most likely to review large volumes of machine text are keyboard users.
- Automated tools catch roughly a third of real issues. We should not report an "accessibility score" as a CI gate; we should fix the specific violations it names.

Text that is not English.

- The engine works in **bytes**; the editor works in UTF-16 code units. Anything crossing that boundary without the mapping layer will corrupt Chinese, Japanese, Korean, Arabic or Indic text — silently, which is the worst kind.
- Our word count under-counts CJK substantially. Search recall for CJK collapses mid-clause because the tokeniser splits on whitespace and these languages do not use it.
- The fix is a character-bigram tokeniser for the search index rather than a dictionary segmenter — cheaper, no large data file, and measurably better recall for our case.
- **The decision to make now:** whether CJK is in scope for v1. In means fixing word count and search. Out means saying so plainly rather than shipping something that quietly does not work. Note that some of the vaults in our own test corpus are Chinese, so we already have real data to test against.

PART V

The honest assessment

37 Strengths, weaknesses, opportunities, threats

I ran twelve separate adversarial audits against our own case. This is what survived.

EXHIBIT 29

Where we actually stand

STRENGTHS	The engine is real, tested against 8,513 foreign files, and correct in a way no competitor is · 88.6% of a shipping editor already exists · the incumbents have the defect and have publicly admitted it · we can ship without permission from anyone · two founders and a team, funded by services rather than a clock
WEAKNESSES	Zero users, zero revenue · no CI in a product about correctness · every distribution channel is borrowed · we own no audience, no list, no store presence · 83% of vaults refused today · our own gates have lied to us · we priced the one thing the category gives away free
OPPORTUNITIES	Slop is 23.7% and growing 149%/year with nobody on it · provenance is uncopyable by any tool that regenerates files · the 501-like live-preview bug is an audience we can buy for one week of work · teams reviewing AI output is a budget that did not exist two years ago
THREATS	Zed or Cursor adding markdown vault semantics — one sprint for them · Obsidian shipping first-party AI editing · the labs making review unnecessary by making output trustworthy · us running out of attention before revenue

38 The twelve audits, in one table

We ran twelve separate adversarial reviews against our own case. Each had to steelman the idea first, rank severity, and say what evidence would change its mind.

EXHIBIT 30

What each audit concluded

#	Angle	Worst finding	Severity
1	The premise	Two mutually exclusive buyers, never chosen between	SEVERE
2	Market fit	The differentiator maps onto almost none of the top complaints	SEVERE
3	Competition	Zed ships our flagship demo already	CRITICAL
4	Features	Most of the feature list has no named user; the table-stakes gap is larger than the novel features	SERIOUS
5	Execution	No CI, four gates blind, engine unwired, estimate 2.5× optimistic	SERIOUS
6	Unit economics	Priced below the category's credibility floor, on the wrong axis entirely	SEVERE
7	D2C	Zero lock-in by design; the category leader is free	SERIOUS
8	B2B	The closest analogue runs a large B2B business with no B2B features	OPEN
9	Capacity	Months went into tooling for ourselves rather than product for a customer	SERIOUS
10	The kill case	Two years of success equals 78 hours a month of consulting	CRITICAL
11	What to build	Not a different product — a different claim on the same code	—
12	The plan	Test for two weeks before building for ten	—

39 The critique in full, audit by audit

Each of the twelve had to steelman our position first, then take it apart, rank severity, and say what evidence would change its mind. These are their findings, not mine.

Audit 1 · The premise — SEVERE

- The chain we rely on is: markdown is the right substrate → byte-exact editing is the differentiator → AI-native developers are the buyer → an editor is the form factor → a subscription is the model. **We have never tested the links separately, and a chain fails at its weakest.**
- Links 1 and 4 hold. Link 2 is true but unfelt. Link 3 is open. **Link 5 is false** — the category gives the editor away.
- The finding that matters: two mutually exclusive buyers, never chosen between.
- *What would change its mind:* evidence that one buyer segment is large enough alone.

Audit 2 · Market fit — SEVERE

- Applied honestly, the Sean Ellis test — would 40% be very disappointed without this? — almost certainly fails today for byte-exactness, because 4 people in 12,556 mentioned the problem at all.
- Kano-classified, most of our feature list is **indifferent**: users neither miss it nor delight in it.

- The category "byte-exact markdown editor" is not one anybody searches for. We compete, as far as a buyer is concerned, in "markdown editor" — where the leader is free.
- *What would change its mind:* provenance testing as a *performance* attribute rather than indifferent.

Audit 3 · Competition — CRITICAL

- Zed ships per-hunk accept/reject, documented, today. Our flagship demo was not novel.
- The asymmetry is stark: Cursor, Zed, GitHub and Anthropic have teams, distribution and capital. Adding markdown vault semantics is roughly one sprint for any of them.
- The "we are more correct" defence has a poor historical record in prosumer tools. Being right is not a moat if nobody can perceive the difference in the first session.
- *What survives:* they cannot do provenance without abandoning whole-file rewriting, which is architectural, not a sprint.

Audit 4 · Features — SERIOUS

- Most of the original feature list could not name a user and a frequency. Those were cut.
- **The table-stakes gap is bigger than the novel-feature gap.** A reviewer who cannot find quick-switch stops writing the review before reaching anything clever.
- Diff review is *work*, and we had been treating it as a feature. A forty-hunk change is a chore.
- Refusal is a "no" to a paying user. If the answer to a refusal is "edit it by hand", we added a step rather than removing one.

Audit 5 · Execution — SERIOUS

- 25,407 lines, 1,575 tests, and **no CI at all** in a product that wants to sell document checking.
- Four of our own gates have reported green while blind. That is a trust problem, not a process one.
- One symbol from one of thirteen engine files reaches product code. The engine is largely not wired in.
- The 41-day estimate assumed full-time work. At our real availability it is roughly **2.5×**.

Audit 6 · Unit economics — SEVERE

- Priced below the category's credibility floor, and on the wrong axis entirely.
- With no paid channel, CAC is founder-hours, and founder-hours have a market price we can look up: ₹1,400–2,000.
- The free tier with hosted AI is the largest uncontrolled cost. BYO key first is the structural answer, not a policy one.

Audit 7 · D2C — SERIOUS

- **We made exit free by design.** Files stay on the user's disk. Ethically right, commercially hostile, and the record had never said so.
- Obsidian is free and beloved. The only thing that has ever successfully charged individuals in this category is sync.
- Developer free-to-paid is the hard half of freemium, and we have zero paying users to calibrate against.

Audit 8 · B2B — OPEN

- The closest analogue runs a substantial B2B business with **zero B2B features**.
- Our genuine B2B asset is architectural: documents live in the customer's repo, so data residency and exit are already solved — normally an enterprise feature costing months.
- The procurement wall (SOC 2, SSO, questionnaires) is real but late. Do not build it before a customer refuses to pay without it.

Audit 9 · Capacity — SERIOUS

- Substantial founder-months went into infrastructure for ourselves — 131 skills, 149 scripts, 69 gates, a book — rather than product for a customer.
- The learning loop those support is not running: 6,884 routing decisions produced one feedback label.
- **2,219,390 words of documentation against 25,407 lines of code — 87 words per line.** That ratio is what someone does when they are uncertain and writing feels like progress.

Audit 10 · The kill case — CRITICAL

- Two years of success equals 78 hours a month of consulting. Stated without rebuttal, as instructed.
- Opportunity cost: the same skills could build something with a shorter path to revenue, and several candidates exist inside our own ecosystem.
- Its one sentence to us: *build the thing that pays now, and let the asset follow if the evidence supports it.*

Audit 11 · What to build

- The reconstruction is not a different product. It is a different **claim** on the same code.
- Nine options scored. Services scored highest and is not a product. **Provenance scored highest among things we could build**, at ~90% reuse.

Audit 12 · The plan

- Test for two weeks before building for ten.
- Rotate the tokens on day one — an action, not a decision.
- Every phase needs an observable outcome, and every bet needs a date by which it is settled.

NOTE

What the critique did not find. No audit concluded the engine was wrong, the code was bad, or the correctness work was wasted. Every severe finding was about the *claim*, the *buyer*, the *price* or the *channel* — never the machine. That is a repairable position.

40

Risks, ranked, with what we do about each

EXHIBIT 31

The risk register

#	Risk	Severity	What we do
1	Provenance turns out to be a nice-to-have	CRITICAL	Two-week test before any code. Kill gate at 4 of 10
2	Zed or Cursor add vault semantics	CRITICAL	One sprint for them. Our answer is depth in markdown, not speed
3	We never solve distribution	CRITICAL	The free plugin is the only owned channel we can build cheaply. Ship it week 2
4	Obsidian ships first-party AI editing	HIGH	We would become a plugin. Plan for that outcome rather than deny it
5	The engine work overruns	HIGH	It is 5 defects and 8 days of wiring, all scoped. Red proofs exist for two
6	Client work crowds out product	HIGH	Cap at 78 hours/month between us. Written down, not assumed
7	Slop stops being a problem	MEDIUM	If models stop producing slop we lose the wedge. Watch the trend quarterly
8	We build for both buyers and serve neither	HIGH	Decide in week 1. It is decision #1 for a reason
9	GST registration triggered unexpectedly	MEDIUM	Reverse charge has no turnover floor — registration starts with the first API purchase
10	Support load with two founders and a team	MEDIUM	Free tier has no SLA. Say so on the page

41

The war-game — what happens when someone bigger moves

Not a risk list. Specific scenarios, what breaks, and whether our response is credible for two founders.

EXHIBIT 32

Six moves, and our answer to each

Scenario	How long we would have	What breaks	What we do	Credible for us?
Obsidian ships first-party AI editing	6–12 months	Our reason to exist as a separate editor	Become excellent <i>inside</i> Obsidian via the plugin, and keep the standalone for teams	Yes — the plugin is already our week-2 move
Zed or Cursor add markdown vault semantics	One sprint for them	Our differentiation narrows to provenance alone	Go deeper on provenance: signing, export, audit. They will not follow into compliance	Partly. It is a real squeeze
GitHub ships an editor over repo markdown	12+ months	They own the substrate, auth and distribution	Be the thing their agent writes <i>into</i> , not a competitor to their editor	Yes, but it caps us
A lab ships a filesystem-backed document surface	Unknowable	Possibly everything	Nothing. This is unhedgeable and we should say so	No
A funded startup ships the same thesis with eight engineers	3–6 months	Our lead	Ship faster on the narrow thing, and lean on the corpus and tests we already have	Partly
Nobody moves and the category never forms	—	The quiet one nobody war-games	This is the most likely failure and the two-week test is aimed directly at it	Yes — it is why we test first

EXHIBIT 33

Our claimed advantages, stress-tested

What we claim	Obsidian moves	Zed adds markdown	A lab ships it
Byte-exact engine	SURVIVES	SURVIVES	SURVIVES
Provenance	SURVIVES	ERODES	GONE

What we claim	Obsidian moves	Zed adds markdown	A lab ships it
Refuse rather than guess	SURVIVES	SURVIVES	ERODES
Reads any existing vault	ERODES	SURVIVES	ERODES
Files never leave the user	SURVIVES	SURVIVES	ERODES
Being small and fast to ship	SURVIVES	ERODES	GONE
Community and ecosystem	GONE	GONE	GONE

Read the last row honestly. We have no community advantage and cannot build one while refusing plugins. Every scenario takes it away because we never had it.

The one that should worry us most is not a competitor. It is scenario six — that byte-exactness and provenance are things we find interesting and the market does not. Every other row has a response. That one only has a test.

42 The three arguments I cannot fully answer

RISK

One. The money. If everything goes right — 502 paying users, 171,200 cumulative visitors, two years — this produces about **₹1,09,135 a month**. That is the same as roughly **78 hours of consulting**, billable next week. Building this is not the fast route to that number. It is the route to owning something that keeps paying after we stop. That is a real reason. It is also the *only* reason, and we should both say it out loud before committing two years.

RISK

Two. We have never picked a buyer. Every plan serves the developer who works with AI all day. Every problem statement that survived our filtering serves someone who is liable when a document is wrong. Speed versus proof. Different products.

RISK

Three. Distribution. Every channel we have belongs to someone else. HN is one shot. The plugin store is Obsidian's. Our name is the generic word for the thing. **We have a product thesis and no distribution thesis**, and the second is harder than the first.

PART VI

How we build it

43 The stack, and why each choice

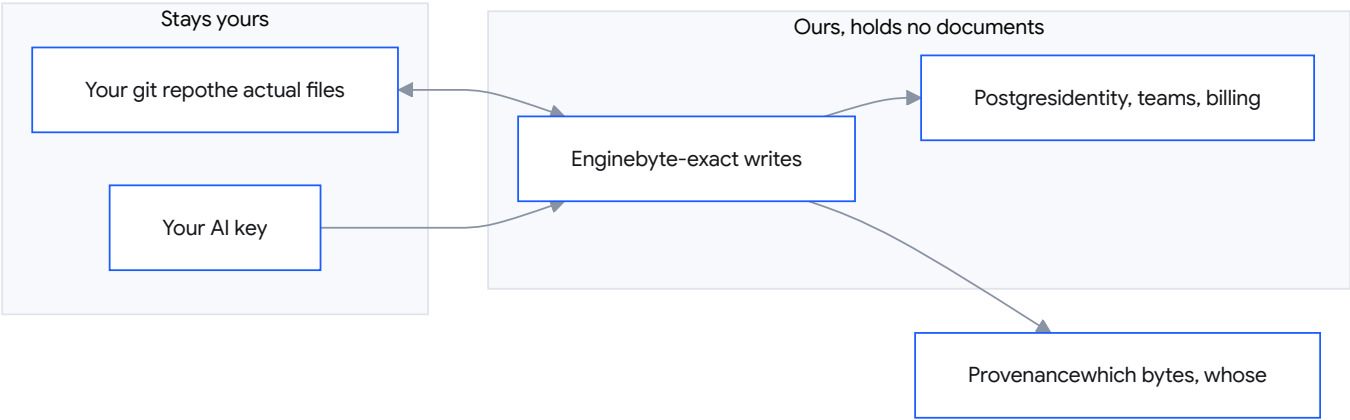
Everything here is chosen for two people plus a team who must operate it without a dedicated ops person.

EXHIBIT 34

The stack

Layer	Choice	Why this one
Frontend	Next.js 16 · React 19 · server components at the edges, one hydrated island for the editor	The editor is irreducibly client-side; everything around it is not. A server round-trip per note switch is indefensible
Editor	CodeMirror 6	Already integrated. The byte-offset boundary is the risky seam and it is already mapped
State	Zustand, three stores	Readable from outside React, which the engine and the desktop bridge both need
Documents	The user's own git repo. Never moves	This is the whole architecture. Data residency, export and trust are solved by never holding the file
Control plane	One Postgres 18, holding zero document bytes	Identity, teams, billing, entitlements, audit. Nothing else
Blobs	Cloudflare R2	Free egress, which is load-bearing at our margins. Note: no object versioning, so DR lives in the key layout
Desktop	Tauri v2	Already scaffolded. Gives real filesystem access without shipping a browser

Layer	Choice	Why this one
Repo access	GitHub App, PR-based writes by default	Proposing a change is architecturally honest for a product whose promise is not corrupting files
AI	User's own key first, ours as an option	Solves our budget problem and their trust problem in one decision
CI	GitHub Actions, ported from the other repo	One day of work. Currently absent



NOTE

The one rule that governs the architecture: we never hold your documents. It is the right thing ethically, it removes most of our legal surface, and it is commercially awkward because it means leaving us costs a user nothing. We accept that trade knowingly.

44 Every layer, decided

The full engineering plan runs to thirteen sections in the record. This is each decision and the reason, so you can argue with any of them.

EXHIBIT 35

The eleven layers

Layer	What we picked	What we rejected, and why
Rendering	Server components at the edges, one hydrated island for the editor	Full client SPA — loses the free public reader and forces an auth waterfall. Server-render per note — a round trip per tab switch is indefensible in an editor
State	Zustand, three stores, plus one deliberate non-store	Redux (ceremony), Jotai (fragments the outbox invariants), Context (re-renders on every keystroke). The EditorView is held in a module singleton because putting a mutable object in a store causes a render loop
Editor core	CodeMirror 6	Already integrated. The real risk is the boundary: the engine speaks bytes , CodeMirror speaks UTF-16 code units . Every offset crosses a mapping layer or we corrupt multi-byte text
API	REST with idempotency keys on every write	A retried commit on flaky mobile must not duplicate a note. Error format is RFC 9457
Data	One Postgres 18 as a control plane, zero document bytes	Supabase rejected: we write RLS and SQL functions by hand anyway, so the abstraction earns nothing. Documents never enter the database — that is the architecture, not an optimisation
Auth	GitHub App, not an OAuth app	An OAuth repo scope asks for everything. The App asks per-repository, and the consent screen is a conversion surface
Writes	PR-based by default	Architecturally honest for a product whose promise is not corrupting files. Direct commits are opt-in
Blobs	Cloudflare R2	Free egress is load-bearing at our margins. Caveat: R2 has no object versioning , so disaster recovery must live in the key layout
Hosting	Vercel for the app, Cloudflare for the edge	Boring, managed, swappable. Two founders cannot run a cluster
Desktop	Tauri v2	Already scaffolded. Today it is a thin wrapper pointing at a hosted URL — that is not a local-first app and turning it into one is real work
Offline	Service worker via @serwist/next	The hand-rolled one already caused a stale-chunk incident. Replace it

Layer	What we picked	What we rejected, and why
AI	User's own key first, ours as a paid option	Solves our budget and their trust in one decision. CORS means a browser may not be able to call some providers directly — this is checked per provider, not assumed
CI	GitHub Actions, ported from the sibling repo	One day. Currently absent entirely

45 The things that will bite us

Named now so they are not surprises.

- **Byte offsets versus UTF-16 offsets.** The single riskiest seam. We already fixed one off-by-three here. Anything that crosses it without going through the mapping layer will corrupt non-English text, and it will do so silently.
- **Serverless connection pooling to Postgres.** A classic trap. Needs a pooler chosen deliberately, not discovered under load.
- **The GitHub App consent screen.** It is a conversion surface. Ask for too much and people bounce before they see the product.
- **R2 has no versioning.** If we overwrite a derived artifact wrongly, it is gone. Recovery has to be designed into how we name keys.
- **Our own CI does not exist.** Four gates in this repo reported green while blind. Until CI runs on a deliberately broken commit and fails, we do not actually know that our checks work.

46 Cost, security, and running it

Concern	Position
Infra cost at 100 users	Near zero. Vercel + Neon free tiers + R2 grants cover it
At 10,000 users	Predictable and small, because we store no documents — the expensive thing in most SaaS is the thing we deliberately do not have
The real cost	Inference, if we host it. Which is why BYO key comes first
Security posture	No arbitrary code execution, ever. No plugins. This is why we can promise the file is safe
Data protection	Documents never leave the user's repo. Most of our compliance surface disappears by construction
Reverse charge	Starts at the first rupee. Buying Claude API access is importing a service; registration is compelled with no turnover floor
On call	Two founders and a team, no rotation. The free tier gets no SLA and we say so publicly

47 Roles, permissions, and sharing

EXHIBIT 36

Who can do what, and how it is enforced

Operation	Our posture	Enforced by
Connect a repository	Defer to GitHub. Only someone with admin on the repo can install the App	GitHub, not us
Write bytes	Intersection, checked at commit time. The commit succeeds only if the acting identity has push rights <i>and</i> our own grant allows it	Both, evaluated together
Read and comment	Ours. Our grants apply to our rendered projection, not to the repo	Us
Commit attribution	Deliberately duplicated. The human's identity goes in the commit trailer even when an agent made the change	Git trailer + our provenance record
Team seats	Ours	Control plane

The principle underneath: we never grant access to a repository that GitHub would not grant. We can only ever be *more* restrictive, never less. That means a security question about us becomes a question about GitHub's model, which is a much easier conversation.

48 Trust, safety, and the abuse surface

Surface	The risk	What we do
Prompt injection in a document	A malicious document instructs the agent to exfiltrate other files	The AI never gets ambient repo access. Every read is scoped to what the user opened

Surface	The risk	What we do
A malicious repository	Someone connects a repo designed to break our parser	Shape gate: 4 MB and 200,000 line ceilings, strict UTF-8 decode, refuse rather than repair
Abuse of hosted AI	One account, an automation loop, a large bill	Hard caps, no background jobs, per-account ceilings
Publishing abuse	If we ever publish, a free indexable page is a spam magnet	Publishing is out of v1. If it returns: paid accounts only, noindex by default, system-assigned slugs
Our own supply chain	A dependency compromise reaching user files	No plugin system at all is the largest single mitigation we have

49 Auth, hosting, caching and the rest of the plumbing

The parts nobody asks about until they break.

EXHIBIT 37

The operational layers

Layer	Decision	The reason, and the trap
Authentication	GitHub App installation, plus NextAuth for session. Delete the second identity path	Two auth paths is two session-fixation surfaces for one operator. We currently have an unused Firebase import in the client bundle
Authorisation	workspace_id on every row, pooled row-level security, from day one	Retrofitting tenancy after launch is the highest-cost change on the board. Not a later decision
Hosting	Vercel for the app, Cloudflare for edge and storage	Boring, managed, swappable. Two founders cannot run a cluster
Environments	Preview per pull request, one staging, one production	Preview environments are where the document gates actually run
Caching	Render cache keyed by content hash; nothing user-specific at the edge	A cache that can serve one user's document to another is the worst bug this product could have. Content-hash keys make it structurally impossible
CDN	Static assets only. Never documents	Documents do not leave the user's machine to be cached
Rate limiting	Per identity and per installation, not per IP	GitHub App installation tokens have their own limits we must live inside
Search	Postgres full-text and trigram server-side; MiniSearch client-side only	We currently ship the whole vault to the client — 77 MB parsed per cold start. That has to stop
Backups	The user's git repo <i>is</i> the backup for documents. Postgres has PITR	The thing normally hardest to back up is the thing we do not hold
Disaster recovery	R2 has no object versioning — recovery must be designed into the key layout	This is a real constraint, discovered by reading their docs rather than assuming
Error tracking	Sentry, with document content scrubbed at the SDK before send	No session replay, ever. The DOM we would be replaying is the user's private document

50 Incremental parsing, and why it matters to a user

A technical decision that shows up as something a person feels.

- Re-parsing a large document on every keystroke is what makes editors feel heavy. Incremental parsing re-parses only what changed.
- The candidate is @lezer/markdown, which CodeMirror already uses. **Vendoring it is the recorded decision** rather than depending on it loosely, because we need to control exactly which constructs it recognises.
- **Why it is not optional:** our provenance spans have to survive edits happening around them. That means knowing precisely which byte ranges moved and by how much, on every keystroke, cheaply.
- **The user-visible measure:** time to first keystroke on a cold start, and typing latency on a 10,000-word document. We publish both.

51 What has to be fixed before anything else

These are not features. They are the reasons the product does not currently work.

#	Problem	Effect today	Size
1	NF-1 — a list item at column zero in frontmatter	83% of real vaults are refused. The front door does not open	4 days
2	NF-3 — a bare carriage-return fence	Silently adds a <i>second</i> frontmatter block. Destroys structure	3 days

#	Problem	Effect today	Size
3	No CI	We have shipped on trust, and four gates have reported green while blind	1 day
4	Engine not wired in	One symbol from one of thirteen files reaches product code	8 days
5	Byte budget is a stub	Literally an echo command, in a product about correctness	2 days

PART VII

The plan

52 Before we write any code — two weeks, zero rupees

This is the highest-value fortnight available to us, and it is the part I most want you to agree to.

EXHIBIT 38

The four tests, and what kills each

#	Test	What it costs	Kill signal
1	Show 10 AI-heavy developers a clickable mock of provenance. Ask what they would pay	3 days	Fewer than 4 of 10 call it useful unprompted
2	Ship the nested-construct live-preview fix as a free Obsidian plugin	4 days	Under 200 installs in 14 days
3	Ask 5 people who bill for documents whether "which part did the machine write" is a real problem	2 days	Nobody has ever been asked for it
4	Three landing pages: free, \$8/seat, \$20/seat. Measure email capture	2 days	No captures at any price

- If **1 and 3 both fail**, the provenance thesis is dead and we should say so in week two rather than month six.
- If **2 succeeds**, we have an audience before we have a product. That has never been true for us before, and it is the cheapest distribution we will ever get.

53 MVP-0 — the proof

Ten weeks elapsed at our real availability, not six. The estimate assumes full-time work and we do not have it.

The demo, in fifteen seconds: open a document an agent has been editing. Machine-written spans are visibly marked. Hover shows the prompt and the model. One key reverts one. `git diff` shows nothing else changed.

Ships	Days
NF-1, NF-2, NF-3 engine fixes	8
Provenance store — byte range, prompt, model, time	6
Provenance rendering — machine spans marked, hover detail	5
One-key revert	3
Engine wired behind every write path	8
Quick-switch, command palette, search	6
CI, with a deliberate red run first	1
Byte budget, real	2

Not in it: sign-up, billing, sync, mobile, publishing, hosted AI, any render.

Exit criterion: ten strangers, their own repositories. **Six of ten say they would keep using it.** Not "like it" — keep it.

54 MVP-1 — the first money

- **Vault-wide refactor.** Rename a tag, a heading, a property key. Every link that will change shows as a reviewable hunk. Refuse when a target is ambiguous. This is the 86-like problem and it is the strongest asked-for capability our engine uniquely enables.
- **The editor stays free forever.** We charge for teams.
- **Team provenance** is the first paid line: shared repos, who-wrote-what across a team, per-seat. The model is Obsidian's commercial licence — \$50/user/year with no enterprise features at all, which is what a nine-figure-logo B2B business actually looks

like in this category.

- The unglamorous half: billing, a support inbox, terms of service, and a way to tell users about a breaking change. **None of these exist and all are required before the first paid signup.**

55 MVP-2 — the moat

- **Sync, done provably safely.** The #1 loved feature and #1 switching trigger. A competitor's sync duplicates sections of files; ours structurally cannot, and we can demonstrate it. This is also the only price this category has ever proven.
- **The free live-preview plugin** in Obsidian's store, permanently. 501 likes on the bug it fixes. It is the cheapest audience available to us and it should ship in week two regardless.

56 Money

Line	Number
Monthly cost, the two of us plus infrastructure	~₹1.09L
Consulting hours that cover it	54–78/month
Hours left for product, per founder	~120/month
Funding model	Services. Four fixed-scope engagements a year at ₹3,00,000
Why not a raise	₹4 Cr is 367 months of our nut, from a fund with a 7-year horizon

TEST

The pricing decision, which inverts our original plan. The editor is free. Teams pay per seat. Sync is a separate paid service later. We charge for the two things this category has proven people pay for, and we give away the thing it has proven they do not.

57 The cost model, and how we avoid a surprise bill

Where money goes	At 100 users	At 10,000 users
Hosting and edge	~free tier	small, predictable
Database	free tier	modest — we store no documents
Object storage	free grants	low, egress is free on our provider
AI inference	the only real cost	the only real risk
Code signing	~\$250/year	same
Support	our hours	the binding constraint

How we make an unexpected bill structurally impossible.

- **Bring your own key is the default.** Their key, their spend, our engine. This removes the risk entirely for the users who choose it.
- **A hard cap, not a soft one,** on any hosted usage. When it is reached we stop and say so, rather than continuing and invoicing.
- **Cheap models for cheap jobs.** Filling in a frontmatter field is not a reasoning task and should never touch a frontier model. Routing by task type is the single biggest lever on cost.
- **Nothing runs in the background.** No ambient AI, no automatic passes over the vault. Every call is something a person asked for, which makes the cost predictable by construction.
- **Measured, not guessed:** in our own usage data, output tokens are 75.8% of spend across 2,333 measured turns. Context is nearly free; generation is the cost. That tells us where to optimise and it is why we cap output length rather than input.

58 Pricing, worked through properly

Our original ₹299/₹599 was set with evidence from zero humans. Here is what the category actually shows.

EXHIBIT 39

What this market charges, opened and dated

Product	Editor	Sync	Teams	What that tells us
Obsidian	Free, no limits	\$4/user/mo	\$50/user/yr commercial	The editor is worth nothing; sync and commercial use are worth money
Notion	—	included	~\$10/user/mo	Bundled, and not comparable
Cursor	—	—	~\$20/user/mo	What developers already pay for AI editing

Product	Editor	Sync	Teams	What that tells us
Claude Code	—	—	inside \$20–200	The budget our buyer already has
Bear / Ulysses / iA Writer	\$15–50/yr	—	—	One-time or cheap annual for a writing tool
GitBook / Mintlify	—	—	\$6.70–\$300+/mo	What teams pay for docs infrastructure

Four conclusions.

- **Charging for the editor is charging for the free thing.** Obsidian settled this and everyone priced around it.
- **₹299 (~\$3.13) is below the credibility floor.** A whole editor for less than one competitor's single add-on does not read as good value. It reads as unserious.
- **The two proven prices in this category are sync (\$4) and a commercial team licence (\$50/user/year).** Those are the two things we should charge for.
- **The buyer already spends \$20–40/month on AI tooling.** We are not asking for a new budget line; we are asking for a share of one that exists.

EXHIBIT 40

What I propose

Tier	Price	What it is
Individual	Free, forever, no limits	The whole editor, provenance included. This is the distribution strategy
Team	\$8/user/month	Shared provenance across a repo, review state visible across people, admin
Sync	\$4/user/month, later	Only when it is provably safe. The one price this category has proven
BYO AI key	Free	Their key, their cost, our engine
Hosted AI	Metered, at cost plus a margin	Optional, and never the default

The arithmetic that matters. At \$8/seat, **114 paying seats** covers the monthly nut of ₹1.09L. That is roughly twenty small teams. It is a much more reachable number than 502 individuals at ₹299, and the churn on a team licence is far lower than on a personal subscription.

TEST

The pricing decision in one line: give away the thing the category has proven is free, and charge for the two things it has proven people pay for. That inverts our original plan and it is better supported by evidence than anything we had.

59 Refunds, cancellation, and what happens to your data

Boring, and it is in the checkout flow, so it must be written before the first paid signup rather than after the first complaint.

Situation	What happens
Cancel a team plan	Runs to the end of the paid period. No pro-rata clawback
Refund request inside 14 days	Granted, no questions. EU withdrawal rights make this mandatory for consumers anyway
After 14 days	Case by case, and generous. The reputational cost of a fight exceeds the money
Payment fails	Retry, then email, then a grace period, then downgrade to free. Never delete anything
Downgrade to free	Team features stop. The editor keeps working and every file stays exactly where it is
Account deleted	We erase identity and billing. Your documents were never ours to delete

The one that matters: downgrading loses you features, never files. A user who stops paying keeps a working editor. That is unusual, it is a consequence of never holding their data, and it should be said out loud in the pricing page.

60 What comes in which plan

EXHIBIT 41

The feature-to-tier map

	Free	Team — \$8/user/mo	Sync — +\$4/user/mo
The whole editor	YES	YES	YES

	Free	Team — \$8/user/mo	Sync — +\$4/user/mo
Provenance: see what the machine wrote	YES	YES	YES
One-key revert	YES	YES	YES
Review state per document	YES	YES	YES
Vault-wide refactor	YES	YES	YES
Bring your own AI key	YES	YES	YES
Unlimited local vaults	YES	YES	YES
Provenance across a shared repo	NO	YES	YES
Who on the team reviewed what	NO	YES	YES
Team admin, seats, roles	NO	YES	YES
Commercial-use licence	NO	YES	YES
Document gates in CI	NO	YES	YES
Multi-device sync, provably safe	NO	NO	YES
Hosted AI, metered	optional	optional	optional
Support	community	published response window	published response window

The reasoning behind the split. Everything an individual needs is free, permanently, including the differentiator. We charge the moment there is a *second person*, because that is where the value changes shape — provenance for one person is a convenience; provenance across a team is a record. And sync is separate because it is the one thing this category has proven people will pay for on its own.

WARN

The obvious objection: we are giving away our differentiator. Yes. Deliberately. Provenance free is what makes anyone try it at all, and a product nobody tries has no team to sell to. The paid thing is not the feature — it is the feature *across people*.

61 B2B and D2C, decided rather than described

The counter-intuitive finding that shaped this. The closest structural analogue to us runs a substantial business selling to companies with no enterprise features whatsoever. Obsidian's entire commercial offering is a \$50/user/year licence; their own FAQ answers the "do I have to pay for commercial use" question and that is essentially the whole product.

That tells us our B2B strategy is probably not SSO, SCIM, audit exports and a SOC 2 report. It is a commercial licence and a clear answer about where the data lives.

EXHIBIT 42

The two motions, honestly

	D2C	B2B
The product	Free editor with provenance	Team provenance, shared review state
Price	₹0	\$8/user/month
Who decides	One person, in a minute	One person, in a week
What they ask	"Is it fast?"	"Where does our data live?"
Our answer	Yes, and it is free	In your own repository. We never hold it
Cost to serve	Nearly zero	Support, and eventually a security questionnaire
Churn	High. Zero lock-in by design	Much lower. It becomes how the team works
Our readiness	Ready after MVP-0	Needs billing, terms, and a support channel

What we can honestly sell to a company today, because the architecture already does it:

- **Data residency, solved.** Their documents never leave their repository. This is normally an enterprise feature that costs months.
- **Exit, solved.** If they stop paying, they keep everything, because we were never holding it. That is unusual and worth saying out loud.

- **An audit trail of machine-written text.** Nobody else can offer this at all.

What will stop a deal, priced:

Blocker	When it bites	Cost to fix
No SOC 2	Above ~50 seats, or any regulated buyer	Months and real money. Defer
No SSO	Around 20+ seats	Weeks. Do it when asked, not before
One-person support expectation	The first incident	A published response window and honesty about the free tier having none
Indian entity selling to EU/US enterprise	Procurement, at larger sizes	A merchant of record handles most of it

NOTE

The recommendation: D2C free to build the audience, B2B paid to build the revenue, and do not build a single enterprise feature until a customer refuses to pay without it. Obsidian's precedent says that can go a very long way.

62 Distribution — our weakest area, stated honestly

Everything above is a product argument. This is the part where I have least to offer, and I would rather say that than dress it up.

What we have today: no email list, no audience, no store presence, no inbound. Every channel we have identified belongs to someone else.

EXHIBIT 43

Channels, and who actually owns them

Channel	Who owns it	Compounds?	Our honest read
The free Obsidian plugin	Obsidian's store, under their rules	Yes	The cheapest audience we will ever buy. 501 likes on the bug it fixes. Ship it in week 2 regardless of everything else
Published pages with SEO	Us, if we build it	Yes, slowly	The only fully owned channel. Long payback
Hacker News	Nobody. One shot	No	Good for a spike, useless as a plan
GitHub presence	Partly ours	Yes	Open-sourcing the engine would earn trust and feed competitors. Real trade, not obvious
Content and writing	Us	Yes	Slow, and it is founder-time we do not have much of
Word of mouth	Earned	Yes	Requires a product people want to talk about. Provenance might be that; byte-exactness never was

The thing that would actually make it spread. A shared artefact that carries a visible mark — a document where anyone can see which parts were machine-written. That is a product feature and a distribution mechanism at the same time, and it is the closest thing to virality this product has.

WARN

Say this out loud: a good product does not find its own users. We have assumed it will. The plugin is the one cheap, ownable move on the table, and it costs about four days.

63 Marketing — who, where, and what we say

We have never written this down and it is our weakest area, so this is a first draft to argue with rather than a plan to execute.

The one-line message, by audience.

Audience	What we say
The developer using Claude Code daily	"See which parts of your file the AI wrote. Undo any of them. Nothing else moves."
A small team	"Know what nobody has reviewed yet, across the whole repo."
An agency or consultancy	"Prove which parts of the deliverable were machine-written."
Someone who just wants a markdown editor	"It is free, it is fast, and it never touches a byte you did not ask it to."

Where these people actually are, ranked by how cheaply we can reach them.

EXHIBIT 44

Channels, honestly rated

Channel	Size / reach	Cost to us	Do we own it?	Verdict
Obsidian plugin store	Every Obsidian user	4 days	No — their rules	Do it in week 2. Best value available
r/ObsidianMD, r/ClaudeAI, r/ChatGPTCoding	Large, active, sceptical	Founder hours	No	Participate honestly; never launch-post
Hacker News	One shot, high variance	One day	No	Save it for something finished
Our own published docs and SEO	Slow to build	Ongoing	Yes	The only fully owned channel. Start now, expect nothing for 6 months
GitHub — the engine as an open library	Developers who will never buy	Weeks	Partly	Trust and credibility; also feeds competitors
Writing about what we learned	Small but compounding	Founder hours	Yes	The research behind this document is genuinely interesting and almost none of it is public
Paid ads	—	Money we do not have	No	Not now, and possibly never

The asset we already have and have not used. We ran 26 research rounds and found things nobody has published — that byte-exactness is almost never discussed, that 89 context-pack products launched in 20 months with a median score of 2, that slop is 23.7% of complaints and rising. **That research is the marketing.** It is honest, it is specific, and it makes the case for the product without selling it.

What we do not do: no launch countdown, no waitlist theatre, no "we are building in public" posting that is really just posting. If we have nothing to show, we say nothing.

64 How users hear from us, and how they reach us

The record found we had **no route to tell a user anything** — not a breaking change, not a price change, not a security incident. That is an operational defect, not a marketing gap.

Message	Trigger	How it reaches them
Security or data incident	Any unauthorised access	Email if we have one, and the in-product notice ledger regardless
Breaking change	Any change to how files are written	In-product notice, two weeks ahead
Price change	Before it takes effect	Email, always, before the next charge
Payment failure	Card declined	Email plus in-product
Team invite	Someone invites them	Email — it is the only contact point
Account recovery	They ask	Email
Release notes	Every release	In-product, dismissible, never a modal

The identity rule. We collect an email **only when an obligation is created** — enabling sync, paying, being invited, or asking for recovery. Never at first run. A user who never gives us an address is a permanently supported state, not a funnel leak.

The in-product notice ledger. A small, quiet list the user can open, where every notice we have ever sent them lives. Not a modal, not a badge that nags. It exists so that "we told you" is verifiable by them, not just by us.

How they reach us: one email address, a published response window, and an honest statement that the free tier has no service commitment. One person on call cannot promise more than that, and promising more is how you get a reputation for silence.

65 How people actually get it, and keep it updated

You asked how users install it. This is the part of a desktop product that quietly decides whether anyone uses it.

Distribution, by surface

Surface	How they get it	What it costs us
Web	Visit a URL. Nothing to install. Works with a GitHub repo	Nothing. This is the trial
macOS desktop	Download a .dmg, drag to Applications	Apple notarisation and a Developer ID certificate — \$99/year and a build step. Without it macOS shows a scary warning and most people stop
Windows desktop	Download an .exe	Code-signing certificate. Roughly \$150–400/year, now requiring a hardware token, and certificate validity has dropped to 460 days
Linux	Applmage or a .deb	Nearly free. Small audience, high goodwill
Obsidian plugin	Their community store, one click	Free, and it is our best channel — but they own the rules
Updates	Tauri's built-in updater, signed	Must be set up correctly from the first release or you cannot ship a fix

The sequence I would follow.

- **Web first.** Zero install friction, and it lets someone try the product in a browser tab before committing.
- **macOS second,** notarised properly. Our audience skews Mac-heavy and an unsigned app reads as unsafe.
- **The Obsidian plugin in parallel,** week two, because it is free distribution and independent of everything else.
- **Windows and Linux when asked for,** not before. The code-signing cost and the hardware-token requirement make Windows the most expensive platform to enter.

WARN

Two things that will catch us out. Code-signing certificates now require a physical hardware token and renew on a shorter cycle than they used to — that is a recurring cost and an operational chore, not a one-off. And the auto-updater has to be signed and configured before the first public build; retrofitting it means asking every early user to manually re-download, which is exactly how you lose them.

66 How we know what we know

So you can judge the evidence rather than take it on trust.

Research rounds	26
Agent reports	109
Words of research banked	~282,000
The full record	137 sections, 338,194 words, 325 pages
Sources opened directly	Hacker News API, Reddit JSON, GitHub API, arXiv, vendor pricing pages, Indian tax and EU regulation primary texts
Things measured on our own machine	The corpus run, the table-editor byte destruction, the trace ledger, the code comparison with sgnk-md

Where the evidence is weak, and you should discount accordingly:

- **Reddit was intermittently blocked.** One round got through, another got 403 on every endpoint. Some of our sentiment data is Hacker News only, which skews toward launches and strong opinions rather than daily use.
- **Zero user interviews.** Not one. Everything about demand is inferred from public complaints. That is why the two-week test exists.
- **Some numbers are still unverified.** We opened primary sources for the twenty-one that mattered most; 62% of those needed correction. The rest of the record has not had that treatment.
- **No pricing has been tested with a human.** ₹299 came from us, not from anyone who would pay it.

NOTE

The most important methodological point. We ran adversarial rounds specifically permitted to refute our own thesis, and they did — twice. That is the reason to trust the rest of it. A research process that never contradicts its sponsor is marketing.

67 What we have not done, and would do next

Being explicit about the edges of this document.

Not done	Why it matters	Cost to close
Zero user interviews	Every demand claim is inferred from public complaints	Two weeks. It is test #1
No pricing tested with a human	₹299 came from us, not a buyer	Included in the two weeks
No visual design work	We have a design system and no screens drawn	One week with the system we already own
Reddit access was intermittent	Some sentiment data is Hacker News only, which skews to launches	Re-run when access is stable
No performance benchmark against Obsidian	Speed is the most-loved attribute in the category and we have not measured ours	Two days
Sync is designed but unproven	It is MVP-2 and the hardest thing on the list	Deliberately deferred
No security review by anyone outside	We are asserting our own safety	A paid review before the first team customer

68 The words we use, defined

Because half of these mean different things to different people.

Term	What we mean by it
Splice	Replacing an exact byte range and nothing else
Provenance	The record of which bytes a machine wrote, with prompt, model and time
Span	One contiguous byte range with a provenance record attached
Reviewed	A human has looked at a span and accepted it. It then renders as ordinary text
Refusal	We could not locate a change unambiguously, so we changed nothing and said why
The corpus	8,513 real markdown files from strangers' public vaults, pinned by checksum
Red proof	A test that fails against the unfixed code. Until it fails, it does not cover the bug
Carrier	How we write something markdown has no syntax for — a callout for prose, a fence for data
The gates	Automated checks over a repo of documents: references resolve, nothing is stale
Vault	A folder of markdown the user already had. We never create one

69 What we measure, and what we refuse to measure

We measure	Why
Time to first keystroke, cold	The most-cited reason people love or leave an editor
Corpus refusal rate	Our front door. Must fall from 83% to near zero
Conflict rate on merges	If we refuse too often we will hear it from users first unless we watch it
Share of a document that is unreviewed machine text	The product's own core metric
Plugin installs	Our only owned channel
Team conversations to conversion	The real B2B funnel
We refuse to measure	Why
Session replay	The DOM we would be replaying is the user's private document
Keystroke-level telemetry	Same reason
Document content, ever	We never hold documents. That is the promise
Vanity metrics — signups, stars, page views	They move without the business moving

70 The ninety days, week by week

Assumes both of us, part-time on product, with client work continuing to fund everything.

EXHIBIT 46

The plan, with an observable outcome every fortnight

Weeks	What happens	Observable outcome
0	Rotate the two access tokens. Answer decision #1: which buyer	Tokens rotated. Buyer written down
1–2	The four tests. Build nothing	10 developer conversations, a plugin shipped, 5 liable-buyer conversations, 3 landing pages live
2	Ship the free live-preview plugin regardless of anything else	Installs measurable by day 14
3	Read the results. Go or no-go on provenance	A written decision, either way
3–4	CI, with a deliberate red run. NF-3, then NF-1	CI fails on a broken commit. Corpus refusals drop from 83%
5–7	Provenance store and rendering	A document shows machine spans
8–9	One-key revert, review state, engine wired behind every write	The demo works end to end
10–12	Table stakes: quick-switch, palette, search. Ten strangers try it	6 of 10 say they would keep it
13	Decide MVP-1 scope on what the ten said	Written scope

71 What we decide, and when we stop

EXHIBIT 47

Decisions with owners

#	Decision	Owner	By
1	Which buyer — the AI-native developer, or the person liable when a document is wrong	Both of us	Week 1
2	Accept provenance as the product, byte-exactness as the mechanism	Both	Week 1
3	Editor free, charge for teams	Both	Week 2
4	The two unrotated access tokens	Sagnik	Today
5	Do we ship the free plugin regardless of test 1	Both	Week 2

EXHIBIT 48

The kill switches, dated

Bet	Falsified when	By
Provenance is wanted	Fewer than 4 of 10 developers call it useful unprompted	Day 14
The 83% refusal is one bug, not a class	First patched corpus run still refuses more than 10 of 7,969	Day 24
Our gates can see	CI passes on a deliberately broken commit	Day 3
The audience is reachable	Under 200 plugin installs in 14 days	Day 14
Anyone pays	60 days with a working checkout and zero non-founder paid signups	Day 150
Teams pay	No team converts after 20 qualified conversations	Day 180

72 The decisions only you can make

#	Decision	Why it is first
1	One founder or two?	Every calendar and cost number depends on it. The record contradicts itself
2	Which buyer — the AI-native developer, or the person who is liable when a document is wrong?	They want different products. We have been building for both
3	Do you accept provenance as the product , and byte-exactness as the mechanism rather than the pitch?	It changes the demo, the copy and the price
4	Editor free, charge for teams and sync?	Inverts the revenue plan

#	Decision	Why it is first
5	The two unrotated PATs	Not a decision. An action. Today

RISK

What would make me tell you to stop. If the two-week tests come back with fewer than 4 of 10 developers calling provenance useful, and nobody who bills for documents has ever been asked for it, then there is no product here — only an engine, a consulting business, and a free plugin that makes people happy. That is not a failure. It is a smaller, truer version of the same work, and it pays better sooner.

73 Everything we are betting on, in one place

EXHIBIT 49

The bets, and the evidence that would settle each

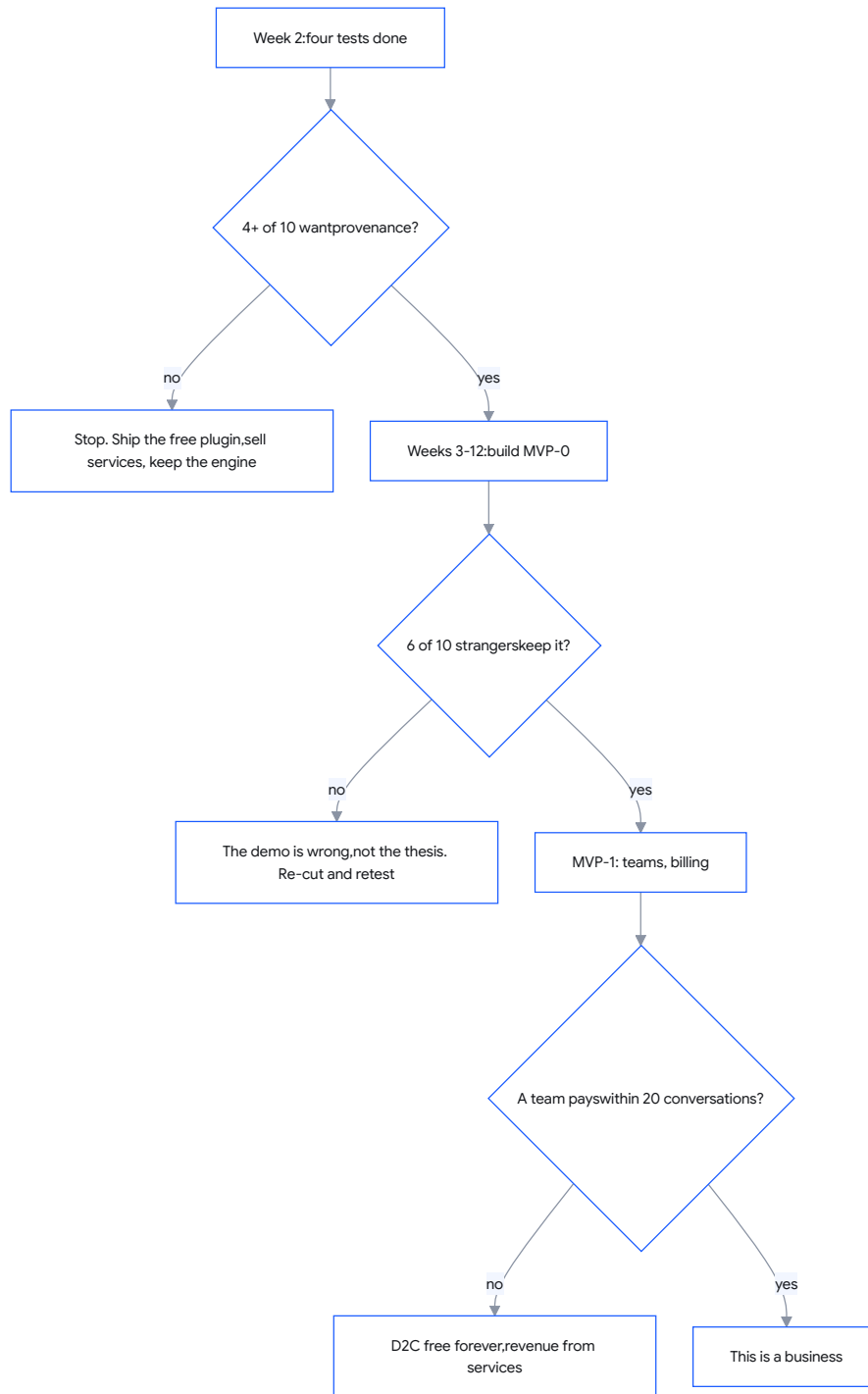
#	The bet	Falsified when	When we know
1	People want to know what the machine wrote	Under 4 of 10 developers say it unprompted	Day 14
2	We can reach anyone at all	Under 200 plugin installs in 14 days	Day 14
3	The 83% refusal is one bug, not a class	Patched corpus still refuses over 10 of 7,969	Day 24
4	Our own gates can see	CI passes a deliberately broken commit	Day 21
5	The demo lands	Under 6 of 10 strangers keep it	Day 84
6	Teams pay	No team converts after 20 qualified conversations	Day 180
7	Slop stays a problem	The complaint rate falls two quarters running	Ongoing
8	We stay funded	Client work exceeds 78 hours/month for two months	Monthly

GOOD

Why I think this is worth doing, stated plainly. We have built the only editing engine in this category that does not damage files, and the incumbents have publicly admitted they have the defect. We were describing it wrongly — as a promise about bytes rather than as a capability nobody else can offer. Provenance is that capability. It answers the fastest-growing complaint in the market, it reuses ninety percent of what exists, and it cannot be copied by anyone who rewrites whole files. The two weeks of tests cost nothing and will tell us whether that reasoning survives contact with real people. If it does not, we will have lost a fortnight and gained the most valuable thing we could have bought.

74 If I am wrong, here is how we will know early

The failure mode I most want to avoid is spending ten weeks and learning nothing. Every phase has a signal that arrives before the money runs out.



Note what the "no" branches are. None of them is "we wasted two years". Every one lands somewhere useful — a free tool people like, an engine that works, a services business already paying our bills. **That asymmetry is the actual argument for trying.**

75 What I actually think

- **The engine is the best thing either of us has built.** It is correct in a way the funded competitors are not, and they have admitted the defect in public.
- **We were selling it wrong.** "We don't corrupt your bytes" is a promise nobody asked for. "You can see which parts the machine wrote, and undo any of them" is the same engine answering the fastest-growing complaint in the market.
- **The two weeks of testing matter more than the ten weeks of building.** If provenance is not wanted, we will know in a fortnight for nothing, and the alternative — an engine, a services business, and a free plugin people love — is not a failure.
- **The thing I am least sure about is distribution,** and I would rather we spend a real week on that question than assume a good product finds its own users. It does not.

GOOD

My recommendation. Run the four tests. Ship the free plugin in week two whatever happens. If provenance clears the bar, build MVP-0 over ten weeks funded by services, keep the editor free, and charge teams. If it does not clear the bar, we will have spent two weeks and learned the most valuable thing available to us.